COMPUTER NETWORKS

A Complete Study Material

In-depth explanations with a diagram for every concept

For GATE & Placement Preparation

Based on the GeeksforGeeks Computer Networks Tutorial

What's inside each chapter

Concept intuition (why it exists) • how it works step by step
• a diagram for every idea • worked examples • trade-offs and key insights • comparison tables

Table of Contents

Chapter 7: Application Layer

Chapter 8: Network Security

Appendix: Quick Reference Formulas & Key Points

Chapter 1

Introduction to Computer Networks

Before we dive into how packets move, addresses get resolved, or connections stay reliable, let's answer the basic questions: What is a network? Why do we need one? How is it classified? This chapter builds the foundation everything else rests on.

1.1 What is a Computer Network?

A **computer network** is a collection of two or more computing devices (nodes) connected by a communication medium so they can exchange data and share resources. That definition sounds trivial until you realize what it actually enables: your phone talking to a server in another continent, a printer being used by five people, video calls, cloud storage, online games — everything modern computing depends on.

A Simple Network: Two Nodes Exchanging Data

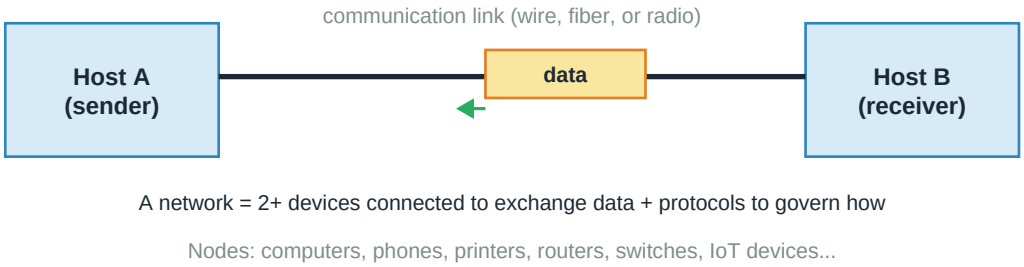


Figure: A minimal network — two nodes, one link, one protocol

A network needs three ingredients: **nodes** (endpoints and intermediate devices), **links** (the medium — cable, fiber, radio), and **protocols** (rules that both ends agree on for framing, addressing, error handling, etc.). Skip any one and communication fails.

Why Networks Matter

- **Resource sharing:** One expensive printer serves many people. Files stored centrally, accessed from anywhere.
- **Communication:** Email, messaging, video calls — humans coordinating over distance.
- **Reliability:** Data replicated across machines survives one machine failing.
- **Scalability:** Add more machines to handle more load, instead of buying one bigger machine.
- **Cost savings:** Small clients + shared servers is cheaper than one big machine per user.

Two Main Design Choices

Client-Server model: One or more powerful **servers** provide services (files, web pages, databases). Many **clients** request them. Simple to design, easy to secure, but the server can be a single point of failure. Almost every web application uses this.

Peer-to-peer (P2P): Every node is both client and server. No central authority. More resilient and can scale well, but harder to secure and coordinate. Used by file-sharing (BitTorrent), some cryptocurrencies (Bitcoin), and mesh messaging apps.

> ***Analogy:*** *Client-server is like a library: one central place lends books to many visitors. P2P is like a book-swap circle among friends — anyone can lend to anyone. The library is easier to run; the circle keeps going even if one member drops out.*

1.2 Types of Networks by Geographic Scope

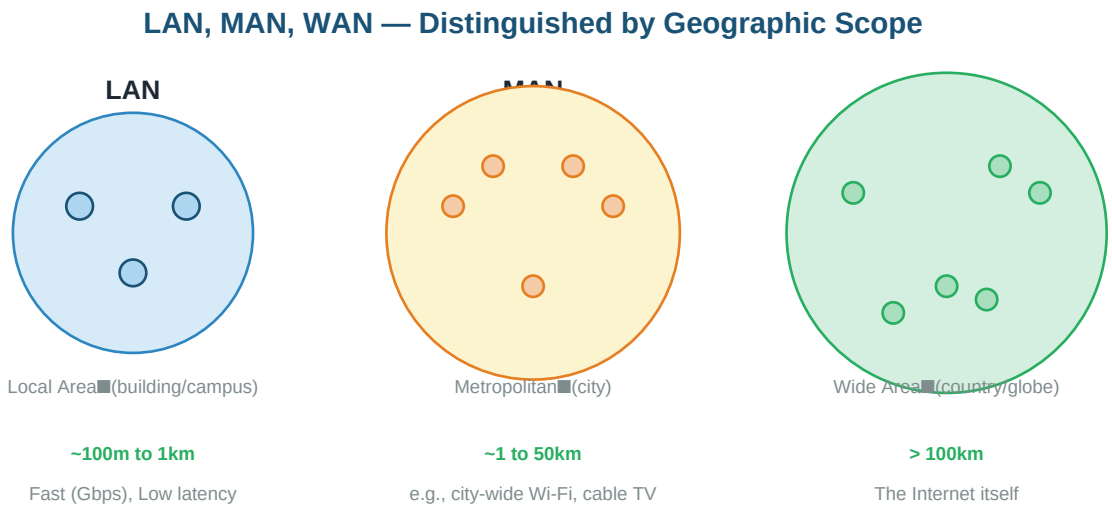


Figure: LAN vs MAN vs WAN — distinguished by how far the network reaches

PAN — Personal Area Network

Devices within a person's immediate reach — typically under 10m. Your laptop connected via Bluetooth to your headphones and mouse. Very short range, low power.

LAN — Local Area Network

A single building, room, or campus. High speed (Gbps), low latency (microseconds), low error rates. Usually owned by one organization. Most commonly implemented as switched Ethernet or Wi-Fi. This is your home network, your office floor, your school lab.

MAN — Metropolitan Area Network

Spans a city or a large campus. Could be a university's fiber connecting all its buildings, or a city government's network linking offices. Typically higher speed than WAN, but slower than LAN.

WAN — Wide Area Network

Spans countries and continents. The **Internet** is the world's largest WAN — literally a network of networks. WANs use a mix of media: fiber for long-distance backbones, satellite for remote areas, cellular for mobile. Higher latency (tens to hundreds of milliseconds) because of physical distance.

Other Networks

- **WLAN:** Wireless LAN — Wi-Fi.
- **SAN:** Storage Area Network — dedicated high-speed network for disk arrays.
- **VPN:** Virtual Private Network — a private connection built through a public network via encryption. Makes remote workers appear as if they're on the office LAN.

1.3 Network Topologies

The **topology** is the physical or logical layout of a network — how the nodes are connected. Different topologies trade off cost, performance, and resilience.

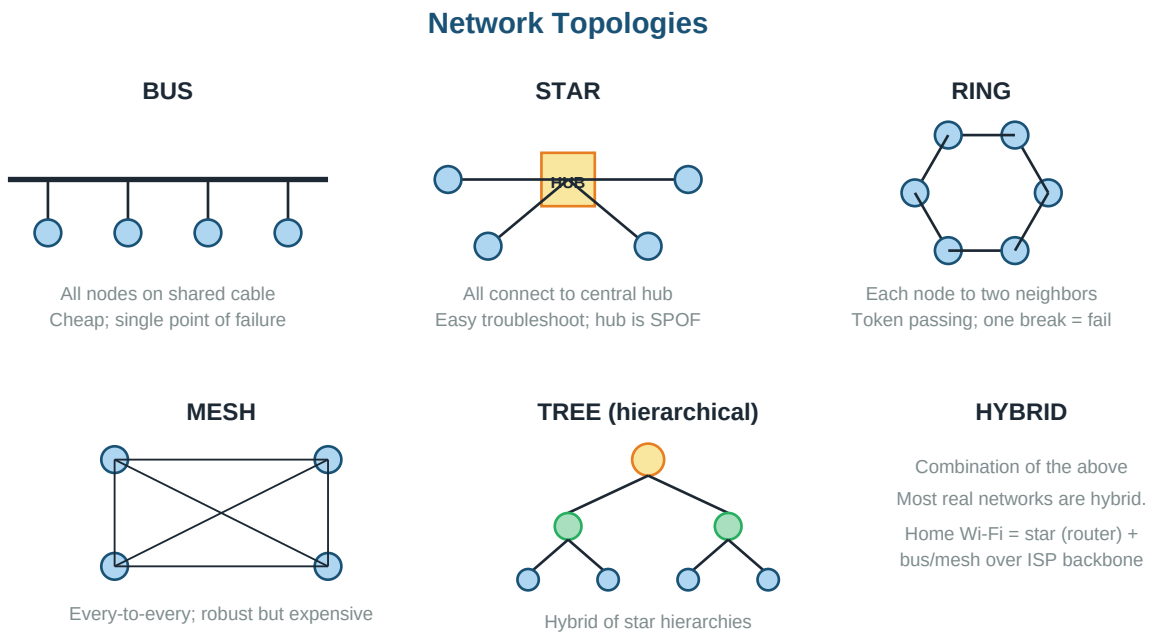


Figure: Common network topologies

Bus Topology

All nodes attached to a single shared cable (the "bus"). When one transmits, everyone hears it — receivers filter for their address. Cheap and simple, but two problems: only one can transmit at a time (contention), and a break in the bus takes down everyone. Used in original Ethernet (10BASE2, 10BASE5).

Star Topology

All nodes connect to a central hub or switch. Easy to add/remove nodes, and one failing cable only affects that node. But the hub is a single point of failure — if the hub dies, so does the network. Most modern LANs use star topology with an Ethernet switch.

Ring Topology

Each node connects to two neighbors, forming a closed loop. Data circulates around the ring. Access is often controlled by **token passing**: a special "token" packet circulates, and only the holder can transmit. Fair and orderly, but any break disrupts the ring (mitigated by dual counter-rotating rings). Used in FDDI and IBM Token Ring — mostly historical now.

Mesh Topology

Every node connected to every other node — full mesh. Extremely robust (multiple paths between any two nodes) but expensive: a network with n nodes needs n(n-1)/2 links. Full mesh is only practical for small critical networks; partial mesh is common in the internet backbone.

Tree Topology

Hierarchical structure — combining star topologies with a bus backbone connecting them. Common in large enterprises where different floors or departments each have their own star, connected by a backbone.

Hybrid Topology

Any combination of the above. Real-world networks are almost always hybrids. Your home has a star (devices → router), your ISP uses tree/mesh, and the internet backbone is a partial mesh.

1.4 Transmission Modes

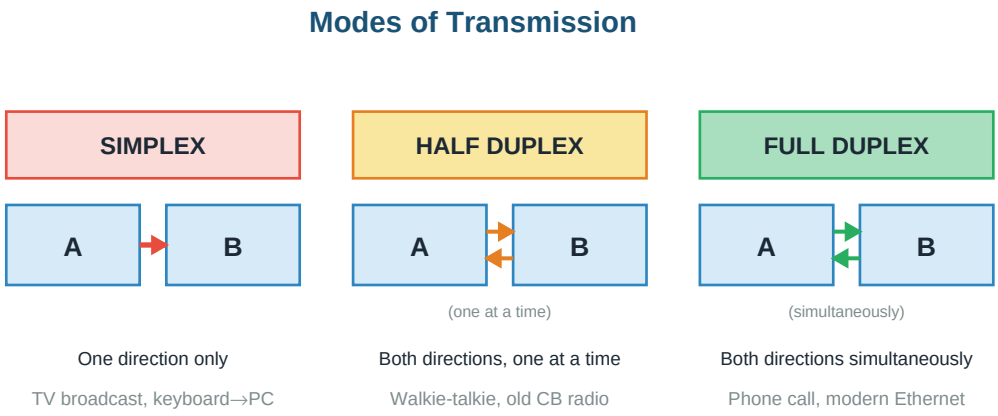


Figure: Simplex, half-duplex, and full-duplex — how many directions and when

- **Simplex:** Data flows in one direction only. TV broadcasting, keyboard → computer. Never a reply.
- **Half-duplex:** Both directions, but only one at a time. Walkie-talkies ("over"). Efficient use of a shared channel; nobody talks over anyone.
- **Full-duplex:** Both directions simultaneously. Phone calls, modern switched Ethernet. Effectively doubles bandwidth but needs separate channels or clever multiplexing.

1.5 Protocols and Standards

A **protocol** is a formal set of rules that governs communication. It specifies the format of messages, the meaning of each field, the order of exchanges, and what to do on errors. Both ends must implement the same protocol; even a small mismatch breaks everything.

A protocol defines three things:

- **Syntax:** Structure and format of the data — what bits go where.

- **Semantics:** What each field means and what actions each side should take.
- **Timing:** When to send, how fast, and how long to wait for a reply.

Standards are protocols that a body has formally agreed on, so different vendors can build compatible products. Major bodies:

- **ISO** — International Organization for Standardization (OSI model)
- **IEEE** — Institute of Electrical and Electronics Engineers (802.x LAN standards)
- **IETF** — Internet Engineering Task Force (RFCs — TCP, IP, HTTP, DNS)
- **W3C** — World Wide Web Consortium (HTML, CSS)
- **ITU** — International Telecommunication Union (telephony, video codecs)

Chapter 2

Network Reference Models

Networks are complex. Building them from scratch would be impossible without a layered approach that splits the problem into manageable pieces. This chapter covers the two dominant reference models — OSI and TCP/IP — and the idea of encapsulation that makes layering work.

2.1 Why Layer the Network?

Imagine trying to write one massive program that handles everything from electrical signals on a cable to displaying a web page. It would be a nightmare — every change would ripple through everything. Layering breaks this monster into independent pieces, each with a clear responsibility.

Layering gives us:

- **Abstraction:** The application doesn't need to know about voltage levels; the physical layer doesn't need to know about HTTP.
- **Modularity:** Change one layer without breaking others. Replace copper with fiber — only the physical layer changes.
- **Interoperability:** Different vendors implement different layers; if they follow the interface, it all works.

Analogy: *Think of the postal system. You write a letter (application). You put it in an envelope with an address (framing). The post office decides how to route it (routing). It goes on trucks, planes, trains (physical transport). Each step has its own rules and workers. The postal system runs because these layers are independent — trucks don't need to read the letter.*

2.2 The OSI Reference Model

The **OSI (Open Systems Interconnection) model** was developed by ISO in the late 1970s. It defines **seven layers**, each with a specific job. In practice, no real system implements exactly seven layers — but the OSI model is the standard teaching framework, and interview questions love it.

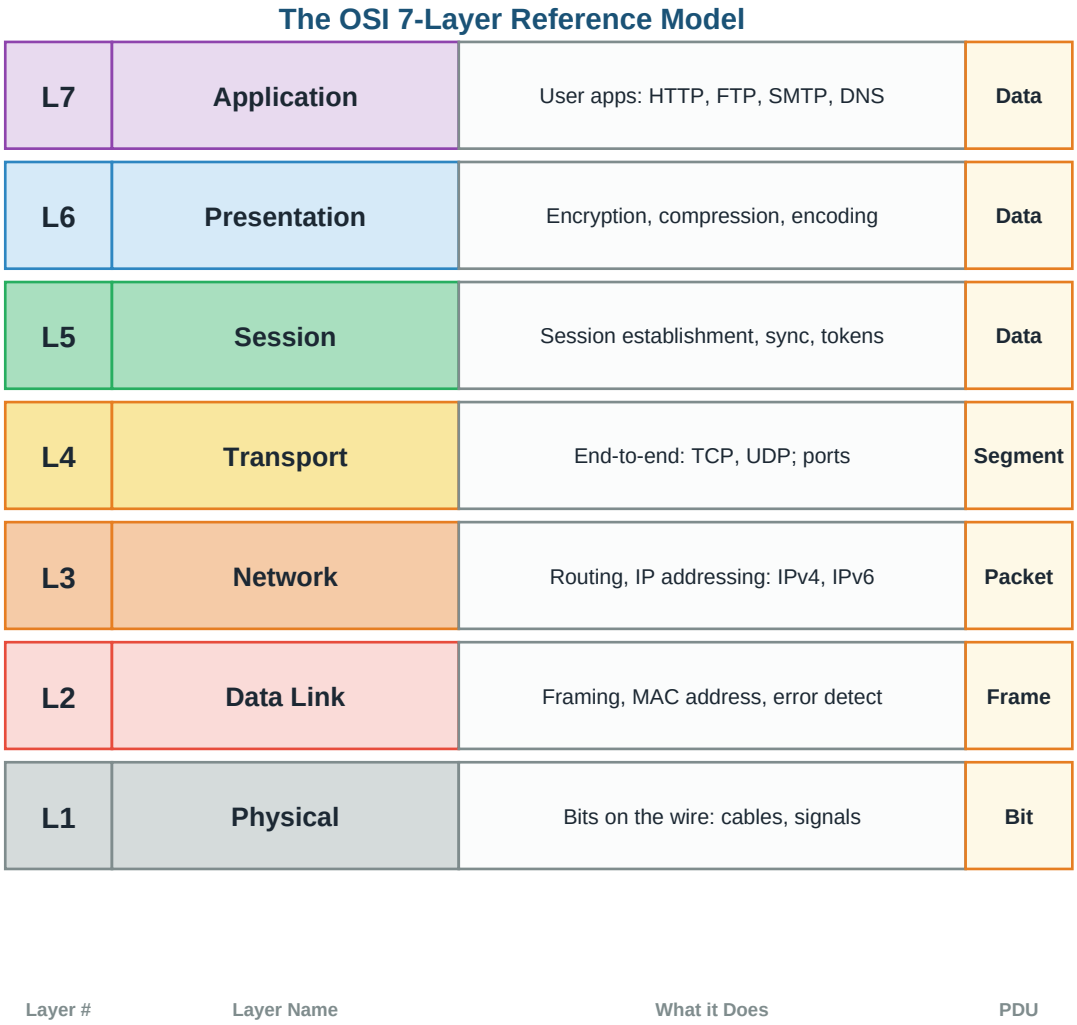


Figure: The OSI 7-layer model with PDU names

Layer 1 — Physical Layer

Moves raw bits from one node to another over a physical medium. Concerned with voltage levels, signal encoding, cable types, connector shapes, and transmission speeds. Standards like RS-232, Ethernet 10BASE-T, and USB live here.

Layer 2 — Data Link Layer

Groups bits into **frames**, adds source/destination MAC addresses, detects errors (CRC), and controls access to the shared medium. Ethernet, Wi-Fi (IEEE 802.11), PPP live here. Divides into two sublayers: **LLC (Logical Link Control)** — framing and error control; **MAC (Medium Access Control)** — who transmits when.

Layer 3 — Network Layer

Routes **packets** across multiple networks. Adds source and destination IP addresses. Handles path selection, forwarding, and fragmentation if the packet is too large for a link. IP, ICMP, and routing protocols (OSPF, BGP) belong here.

Layer 4 — Transport Layer

Provides end-to-end delivery between applications on two hosts. Splits data into **segments**, adds source and destination **ports**. TCP gives reliable ordered delivery; UDP gives fast unreliable delivery. This is where reliability and flow control live.

Layer 5 — Session Layer

Establishes, manages, and terminates **sessions** between applications. Handles synchronization checkpoints, dialog control (who talks when), and session recovery after failures. Rarely implemented as a distinct layer in practice — its jobs are often absorbed by the transport layer or the application.

Layer 6 — Presentation Layer

Translates data between the format an application uses and a common network format. Handles character encoding (ASCII vs Unicode), data compression (gzip), and encryption (TLS). Again, mostly absorbed into the application in practice.

Layer 7 — Application Layer

The layer users see. HTTP, FTP, SMTP, DNS all live here. Provides high-level services like file transfer, email, and web browsing directly to applications.

Key Insight: A common memory aid: **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way — Physical, Data Link, Network, Transport, Session, Presentation, Application (bottom to top).

2.3 The TCP/IP Model

The **TCP/IP model** came earlier (1970s) and describes how the internet actually works. It's more practical and less formal than OSI. Different sources describe it as 4 or 5 layers — we'll use 5 because it maps cleanly to OSI.

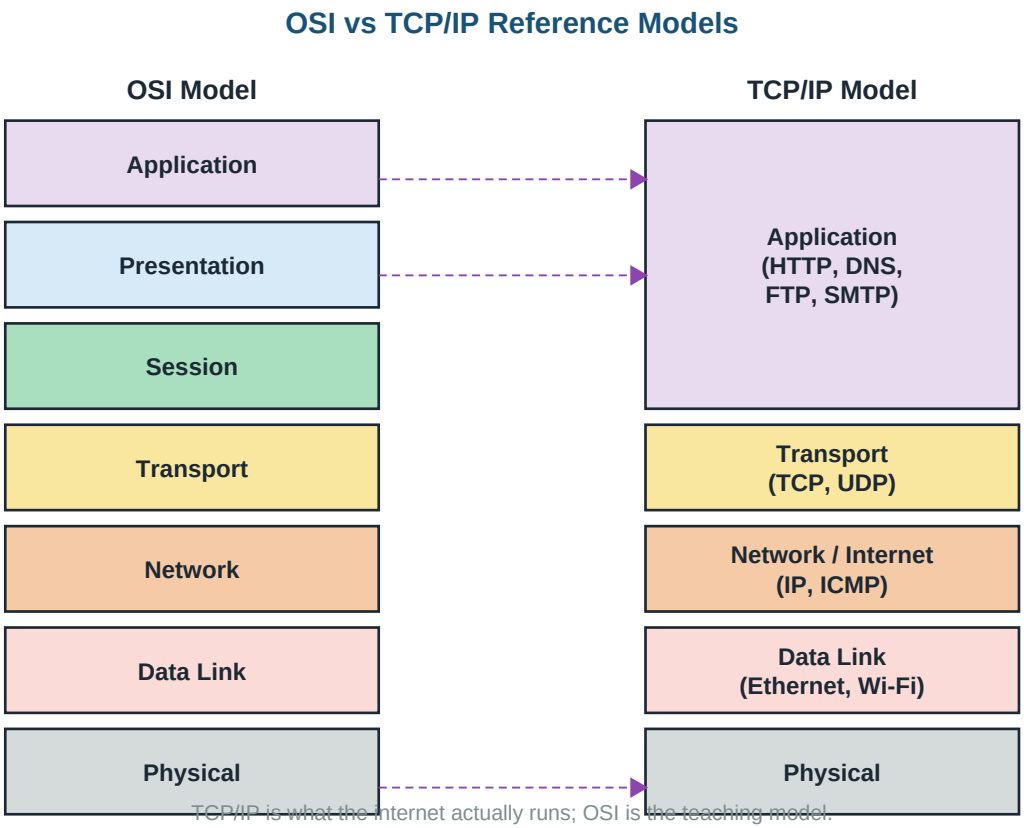


Figure: OSI vs TCP/IP — TCP/IP condenses OSI's top three layers into one

The main differences from OSI:

- **Application (L5-L7 combined):** HTTP, DNS, FTP, SMTP. No separate session/presentation layers — applications handle those concerns themselves.
- **Transport (L4):** TCP, UDP. Same as OSI.
- **Network/Internet (L3):** IP, ICMP. Same as OSI.
- **Data Link (L2):** Ethernet, Wi-Fi. Same as OSI.
- **Physical (L1):** Cables, signals. Same as OSI.

OSI vs TCP/IP — Comparison

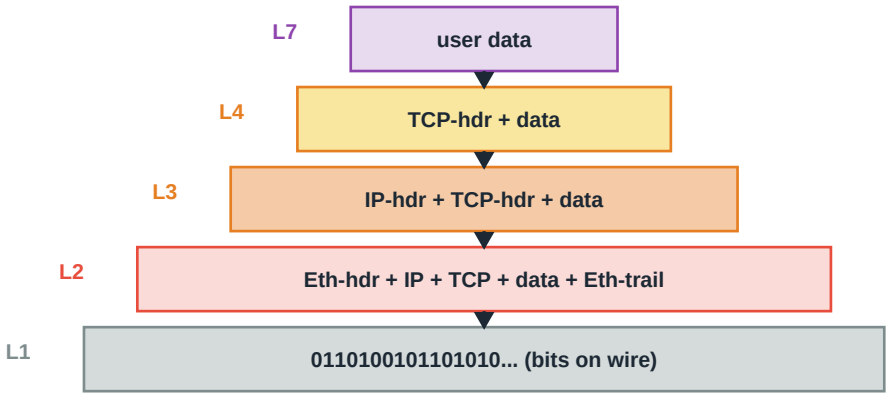
Aspect	OSI Model	TCP/IP Model
Origin	ISO, theoretical	DARPA, practical
Layers	7	4 or 5
Adoption	Reference only	Actually running the internet
L5-L7	Distinct layers	Merged as 'Application'
Protocol independence	Model then protocols	Protocols first, model second

2.4 Encapsulation and Decapsulation

Layering has a magical property: each layer treats the data from the layer above as an opaque payload and adds its own header (and sometimes trailer). This process is called **encapsulation**.

Encapsulation: Each Layer Wraps the Data

Sender side (going down the stack):



Each layer adds its own header (and DL adds a trailer too)

The receiver reverses this: strip headers going up the stack (de-encapsulation)

L7 → L4 → L3 → L2 → L1 on the sender

L1 → L2 → L3 → L4 → L7 on the receiver

Figure: Encapsulation — each layer wraps the previous layer's output

On the sender's side, data flows DOWN the stack: user data → segment → packet → frame → bits. Each layer prepends its header. On the receiver's side, data flows UP: bits → frame → packet → segment → user data. Each layer strips its header, does its checks, and passes up.

The names of the Protocol Data Units (PDUs) at each layer are worth memorizing:

- **Application:** Data / Message
- **Transport:** Segment (TCP) or Datagram (UDP)
- **Network:** Packet (also called Datagram)
- **Data Link:** Frame
- **Physical:** Bit

Chapter 3

Physical Layer

The physical layer is where abstractions meet reality — where bits become voltages, light pulses, or radio waves. It sounds unglamorous but everything else in the stack depends on it working reliably.

3.1 Key Concepts

The physical layer's job is simple in principle: move bits from one node to another over some medium. The hard part is doing it fast, reliably, and cheaply. To understand physical layer trade-offs, you need to understand three fundamental measurements.

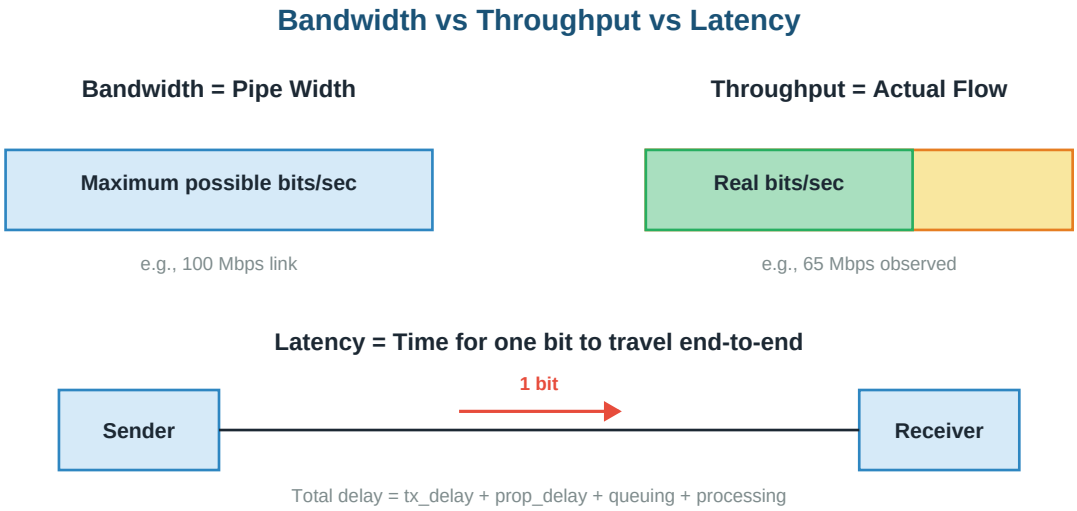


Figure: Bandwidth, throughput, and latency — different things, often confused

- **Bandwidth (B):** The theoretical maximum data rate a link can carry, measured in bits per second (bps, Mbps, Gbps). Determined by the physical medium and encoding. "100 Mbps Ethernet" means bandwidth is 100 Mbps.
- **Throughput:** The actual data rate observed. Usually less than bandwidth due to overhead, congestion, retransmissions, and protocol inefficiencies. A 100 Mbps link often delivers 65-95 Mbps of usable throughput.
- **Latency (delay):** Time for one bit to travel end-to-end. Made of several components (below).
- **Bandwidth-Delay Product (BDP):** B × delay = amount of data "in flight" at any moment. Critical for sizing buffers and windows.

Types of Delay

```
Total delay = t_trans + t_prop + t_proc + t_queue

Transmission delay t_trans = frame_size / bandwidth
(time to push all bits onto the link)
```

```
Propagation delay t_prop = distance / propagation_speed
(time for a bit to travel across the link)

Processing delay t_proc = time to examine header at each node
Queuing delay t_queue = time waiting in router buffer
```

Example: 1 KB frame, 1 Mbps link, 5000 km fiber (v ≈ 2×10■ m/s):
t_trans = 8000 bits / 10■ bps = 8 ms
t_prop = 5×10■ m / 2×10■ m/s = 25 ms
For long-distance, propagation dominates. For short high-speed links, transmission dominates.

3.2 Transmission Media

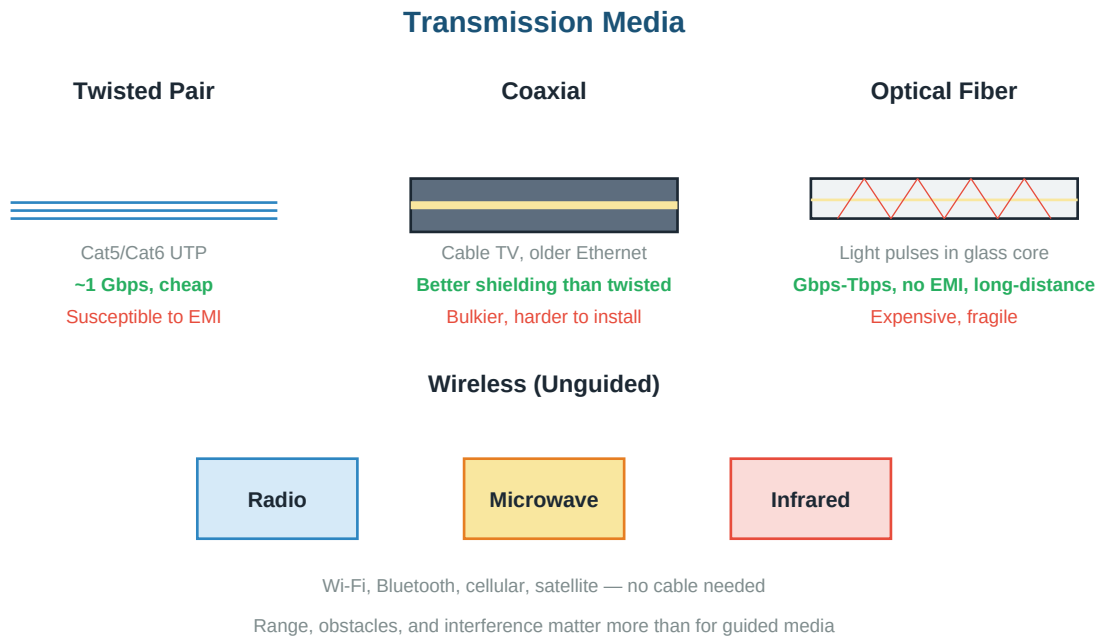


Figure: Guided (wired) and unguided (wireless) transmission media

Guided Media — Signals in a Cable

Twisted Pair

Two insulated copper wires twisted together. Twisting cancels out electromagnetic interference. Comes in **UTP (unshielded)** and **STP (shielded)** flavors. Category ratings: Cat5e up to 1 Gbps at 100m, Cat6 up to 10 Gbps at 55m, Cat6a up to 10 Gbps at 100m. Cheap, easy to install — the workhorse of LANs.

Coaxial Cable

Central conductor surrounded by insulator, mesh shield, and outer jacket. Better shielded than twisted pair, supports higher frequencies. Historically used for early Ethernet (10BASE2) and cable TV. Now mostly displaced by twisted pair (in LANs) and fiber (in backbones).

Optical Fiber

Thin glass strand carrying light pulses. Two types: **single-mode** (very thin core, one light path, long distances, most bandwidth) and **multi-mode** (thicker core, multiple paths, shorter distances, cheaper). No

electromagnetic interference, negligible signal loss over kilometers, enormous bandwidth (Tbps possible). Expensive to install; fragile. The backbone of the modern internet.

Unguided Media — Radio Waves

No cable — signals travel through air or space as electromagnetic waves. Frequency determines behavior:

- **Radio waves (3 kHz - 1 GHz):** Omnidirectional, penetrate walls, long range. AM/FM radio, cellular, Wi-Fi.
- **Microwaves (1 - 300 GHz):** Higher bandwidth, but line-of-sight required. Satellite links, point-to-point relays, 5G.
- **Infrared (300 GHz - 400 THz):** Very short range, blocked by walls. TV remotes, some IoT.

Key Insight: Higher frequency → higher bandwidth → shorter range. This is why 5 GHz Wi-Fi is faster than 2.4 GHz but doesn't go through walls as well.

3.3 Multiplexing

Multiplexing lets multiple signals share the same physical link. Without it, you'd need a separate cable for every conversation. There are three main techniques.

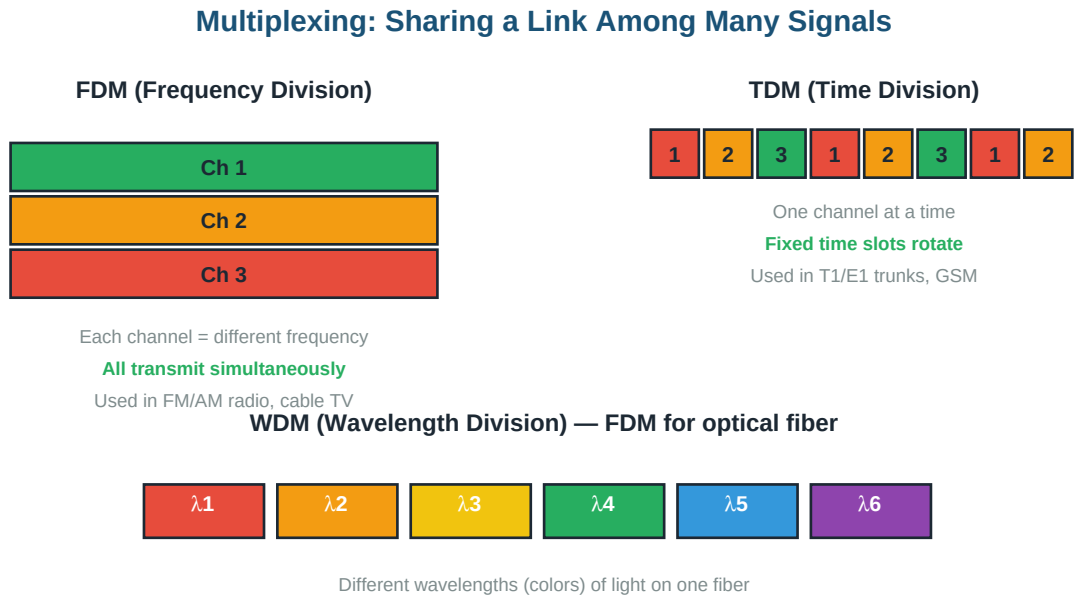


Figure: FDM, TDM, WDM — three ways to share one link

FDM — Frequency Division Multiplexing

Divide the available bandwidth into non-overlapping frequency bands. Each signal gets its own band. All signals transmit simultaneously. Used in radio broadcasting (each station has a frequency), cable TV, DSL. Guard bands prevent interference between adjacent channels.

TDM — Time Division Multiplexing

The whole bandwidth is used by one signal at a time. Each signal gets a fixed time slot in a repeating cycle. **Synchronous TDM** gives each source a slot whether it has data or not (wastes capacity if silent). **Statistical TDM** assigns slots dynamically based on activity — much more efficient. Used in digital telephony (T1/E1),

GSM.

WDM — Wavelength Division Multiplexing

Essentially FDM for optical fiber. Different wavelengths (colors) of light travel simultaneously in the same fiber. **DWDM** (Dense WDM) packs dozens of wavelengths for terabit backbones. This is why one fiber can carry more data every year — new WDM equipment adds more wavelengths without digging up cables.

CDMA — Code Division Multiple Access

A more sophisticated approach. All users transmit simultaneously in the same frequency, but each uses a unique code (pseudorandom sequence). The receiver correlates the incoming signal with the specific code to extract that user's data. Used in 3G cellular networks.

3.4 Switching Techniques

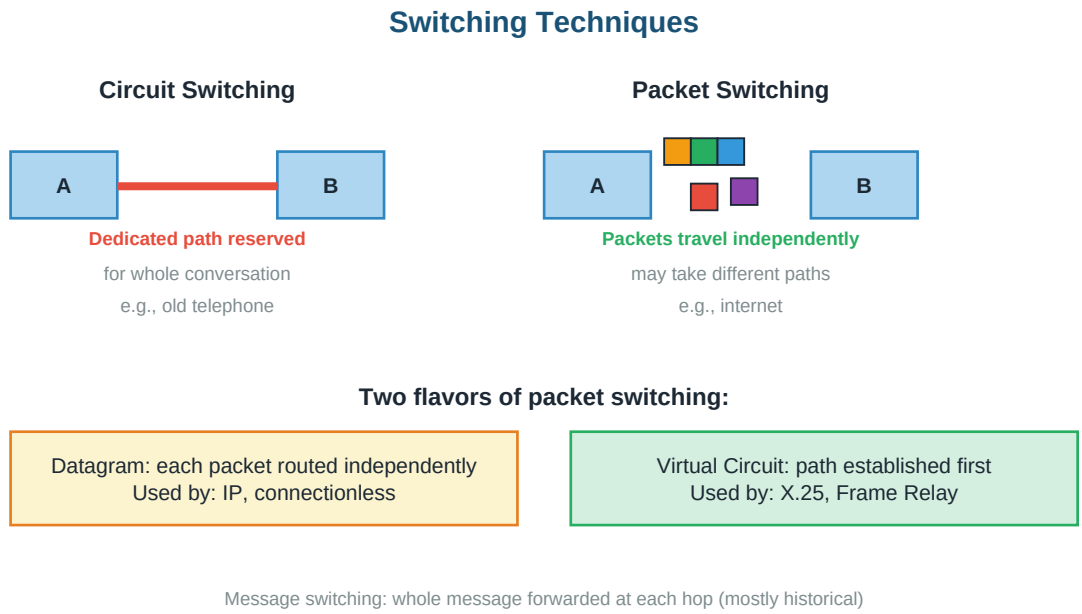


Figure: Circuit vs packet switching — how data actually gets from A to B

Circuit Switching

Before any data flows, a dedicated end-to-end path is established through the network. This path is reserved for the entire duration of the conversation. Resources are guaranteed, latency is predictable, and data arrives in order. But bandwidth is wasted during silent periods.

Classic example: old telephone networks. When you dialed, switches physically connected your line through to the other end for the whole call. Great for voice (constant bandwidth), terrible for bursty computer traffic.

Packet Switching

Data is broken into **packets**. Each packet is routed independently — same source and destination, but potentially different paths. No dedicated resources; the network is shared statistically. This is how the internet works.

Two flavors of packet switching:

- **Datagram (connectionless):** Each packet routed independently. Simple, robust — routes can change on the fly around failures. This is IP.
- **Virtual Circuit (connection-oriented):** A path is set up at the start; all packets follow it. Combines statistical multiplexing with route consistency. Used by X.25, Frame Relay, MPLS.

Message Switching

The whole message is stored at each intermediate node and forwarded as a unit. Doesn't scale — a large message can block a node for a long time. Mostly historical; email server-to-server relay was once like this.

Feature	Circuit	Datagram	Virtual Circuit
Setup	Required	None	Required
Path	Fixed	Per-packet	Fixed
Order	In order	Can be reordered	In order
Reliability	High (dedicated)	Best effort	Better than datagram
Efficiency	Poor for bursty	Good	Good

3.5 Line Coding / Digital Encoding

Bits are logical 0s and 1s. To send them over a wire, we need to represent them as physical signals — voltage levels, light pulses, or radio wave patterns. A **line coding scheme** is the mapping.

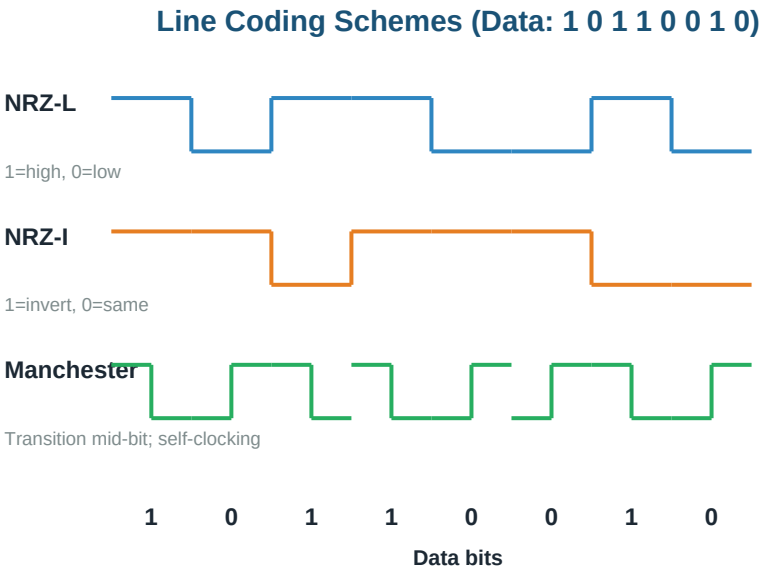


Figure: Common line coding schemes — same data, different signals

NRZ-L (Non-Return-to-Zero Level)

Simplest: high voltage = 1, low voltage = 0. Signal doesn't return to zero between bits. Simple to implement, but a long run of the same bit gives a constant voltage — the receiver can lose sync (no transitions to lock

onto).

NRZ-I (Non-Return-to-Zero Inverted)

Transition = 1, no transition = 0 (or vice versa). Better than NRZ-L for handling long runs of 1s, but a long run of 0s still causes trouble.

Manchester Encoding

Every bit has a transition in the middle. Low-to-high transition = 0, high-to-low = 1. Always self-clocking (the receiver locks onto the mid-bit transitions), but uses twice the bandwidth of NRZ. Used in classic 10 Mbps Ethernet.

Differential Manchester

Always a mid-bit transition. The **presence or absence** of a transition at the start of a bit signals the value: transition at start = 0, no transition = 1. Used in Token Ring.

Key Insight: When picking a line coding scheme, considerations are: **DC balance** (equal 1s and 0s to keep average voltage zero), **clock recovery** (transitions to sync on), and **bandwidth efficiency**. Faster protocols use more complex schemes (4B/5B, 8B/10B, PAM) that trade complexity for efficiency.

Chapter 4

Data Link Layer

The physical layer moves bits. The data link layer groups them into frames, detects errors, and arbitrates who transmits when. It's the layer that turns a raw signal into something usable.

4.1 Framing

The physical layer delivers a stream of bits. But how do we know where one frame ends and another begins? That's **framing** — grouping bits into discrete units with clear boundaries.

Framing: Turning a Bitstream into Discrete Frames

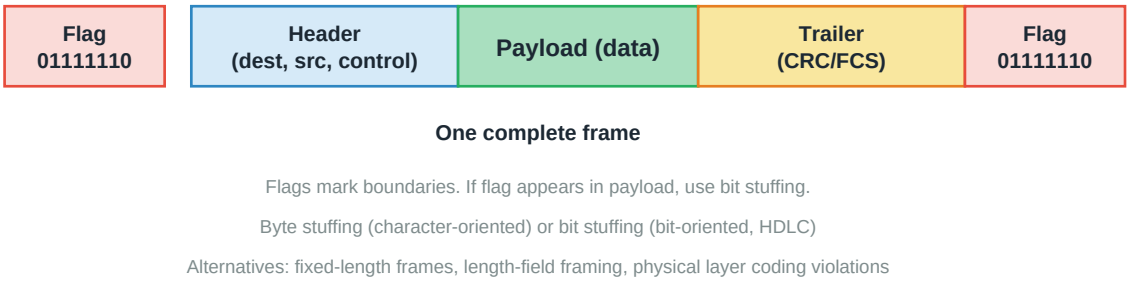


Figure: A typical frame — header, payload, trailer, delimited by flags

Framing Methods

- **Character count:** Header includes a length field. Simple but fragile — a single bit error in the count and everything falls apart.
- **Flag byte with byte stuffing:** Special byte (like 0x7E) marks frame boundaries. If the flag byte appears in data, escape it (like a backslash in code).
- **Bit stuffing:** Use a special bit pattern (like 01111110) as flag. To prevent this pattern in data, insert a 0 after any five consecutive 1s in the payload. Receiver reverses the operation. Used by HDLC and PPP.
- **Physical layer coding violations:** Use signal patterns that never occur in normal data — the receiver detects boundaries by the violation. Used in Manchester encoding schemes.

4.2 Error Detection and Correction

Signals get corrupted. Cables pick up noise, wireless signals fade, cosmic rays flip bits. The data link layer detects (and sometimes corrects) errors so upper layers see clean data.

Parity Check

Simplest error detection. Add one extra bit so the total number of 1s is even (even parity) or odd (odd parity). If a single bit flips, the parity is wrong and we detect it. But two flips cancel out — detects only odd-number errors.

> **Example:** Data 1011001 has four 1s. Even parity → append 0 → 10110010. If any single bit flips, the count of 1s becomes odd → error detected.

Two-Dimensional Parity

Arrange data in a rectangle; compute parity for each row and each column. Can detect all 1-, 2-, and 3-bit errors and correct 1-bit errors (row and column parity both wrong identify the bit).

Checksum

Sum up all the data as integers, then send the (one's complement of the) sum. Receiver adds everything including the checksum — should be all 1s. Detects most errors but weaker than CRC. Used by TCP, UDP, IP.

CRC — Cyclic Redundancy Check

The workhorse of error detection. Treats data as a polynomial and divides it by a fixed **generator polynomial G(x)**. The remainder becomes the CRC. Receiver divides again — remainder 0 means no detected error. Extremely good at catching burst errors.

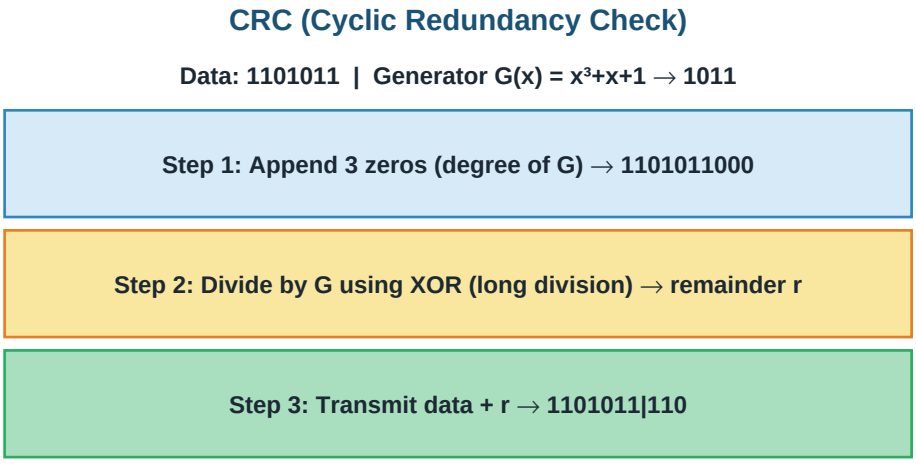


Figure: CRC computation — polynomial division with XOR

Common generators: **CRC-16** (data links), **CRC-32** (Ethernet). CRC-32 catches essentially all real-world errors.

Hamming Code (Error Correction)

Instead of just detecting errors, Hamming codes can **correct** them. Insert parity bits at positions 2■, 2^1, 2^2, 2^3, ..., each covering a specific pattern of data bits. When a bit flips, the failing parity bits pinpoint exactly which position was corrupted.

A Hamming(7,4) code has 4 data bits + 3 parity bits and can correct any single-bit error. Used where retransmission is impossible or too costly (deep-space communication, ECC memory).

4.3 Flow Control and Reliable Delivery

The sender must not overwhelm the receiver. **Flow control** matches transmission rate to what the receiver can handle. Combined with **ARQ (Automatic Repeat reQuest)** for reliable delivery.

Stop-and-Wait

The simplest reliable protocol. Sender transmits one frame, then waits for an **ACK** (acknowledgment) before sending the next. If no ACK arrives before a timer expires, retransmit. Uses a 1-bit sequence number to detect duplicates.

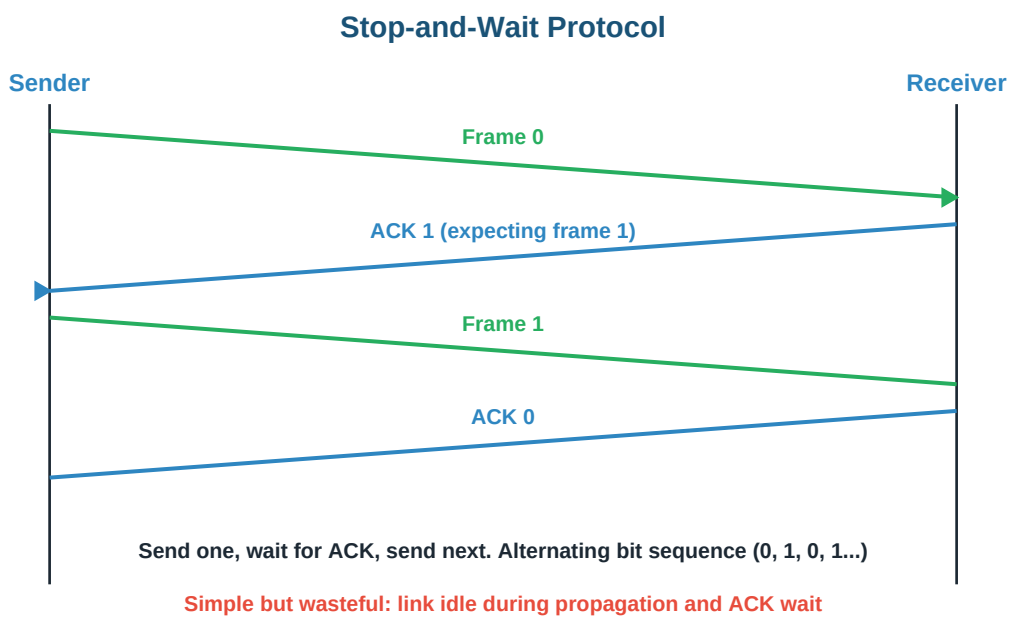


Figure: Stop-and-Wait — one frame at a time

> **Important:** Stop-and-wait wastes the link during the RTT. For a satellite link with 500ms RTT and 1 Mbps, sending 1 KB frames gives an efficiency of about 1.6% — the link sits idle 98% of the time waiting for ACKs.

Sliding Window

Allow the sender to have multiple unacknowledged frames "in flight" at once — up to a window size N. The link stays busy while ACKs come back. Sequence numbers cycle through 0, 1, 2, ..., 2■-1.

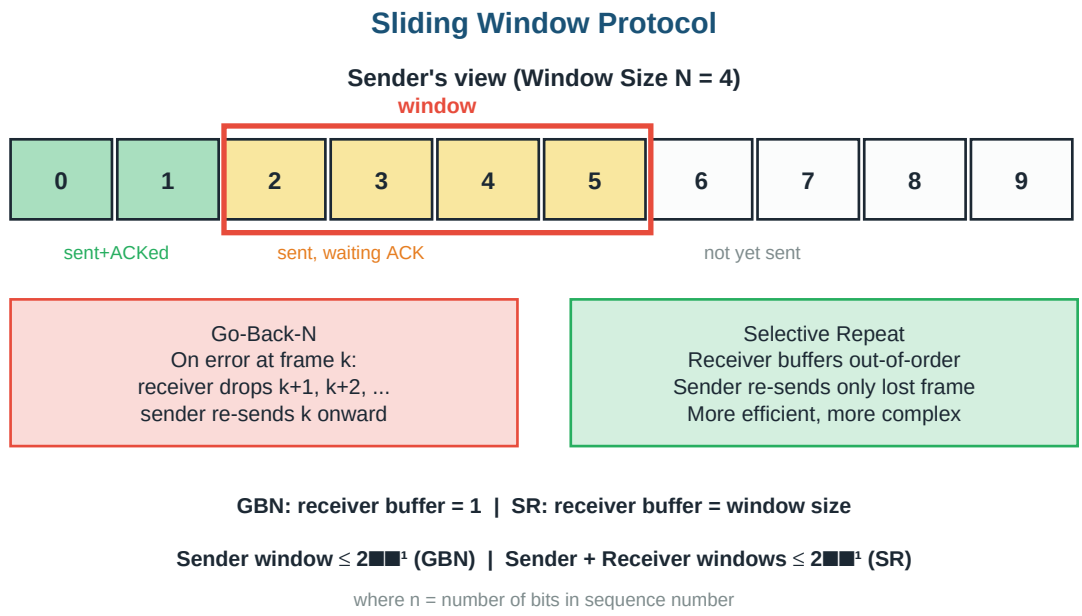


Figure: Sliding window — multiple frames in flight for higher throughput

Go-Back-N (GBN)

Receiver only accepts frames in strict order. If frame k is lost, receiver discards frames k+1, k+2, ... until k is retransmitted. Sender, on timeout, re-sends everything from k onward. Simple; wasteful on lossy links.

Selective Repeat (SR)

Receiver buffers out-of-order frames and sends selective ACKs. Sender only retransmits the specific lost frame. More efficient but more complex, and requires the receiver to have buffer space.

```
Constraint on window sizes (n = seq number bits):
Go-Back-N: W_sender ≤ 2■ − 1
Selective Repeat: W_sender + W_receiver ≤ 2■
(equivalently W ≤ 2■■¹ for symmetric)
```

4.4 Medium Access Control (MAC)

When multiple nodes share the same physical medium, how do we prevent them from all transmitting at once and corrupting each other's signals? This is the **MAC (Medium Access Control)** problem.

ALOHA (Pure ALOHA)

The historically-first random access protocol (developed for radio in Hawaii, 1970s). Rule: if you have data, just transmit. If there's a collision, wait a random time and retransmit. Simple but wasteful — maximum channel utilization is only about 18%.

```
Pure ALOHA throughput: S = G × e■²■ (max ≈ 0.184 at G = 0.5)
Slotted ALOHA: S = G × e■■ (max ≈ 0.368 at G = 1.0)
```

Slotted ALOHA — transmit only at the start of fixed time slots — doubles the maximum efficiency by eliminating the risk of transmitting mid-slot.

CSMA (Carrier Sense Multiple Access)

Improvement over ALOHA: **listen before you transmit**. If the channel is busy, wait. If free, transmit. Variants:

- **1-persistent:** Wait until free, then transmit immediately. High collision rate under load.
- **p-persistent:** Wait until free, then transmit with probability p. Better under load.
- **Non-persistent:** If busy, back off a random time before rechecking. Reduces collisions but adds delay.

CSMA/CD — CSMA with Collision Detection

The evolution used in **classic Ethernet**. While transmitting, also monitor the channel. If a collision is detected (signal doesn't match what was sent), stop, send a **jam signal**, and back off exponentially.

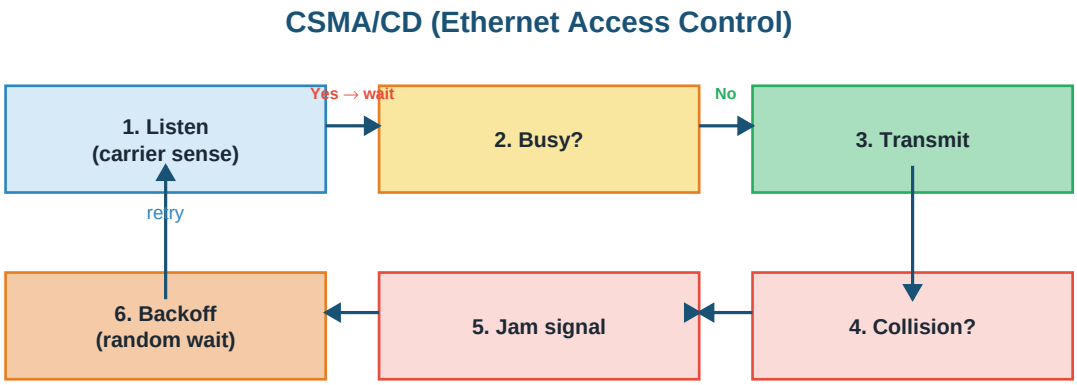


Figure: CSMA/CD — carrier sense, transmit, detect collision, back off

The exponential backoff (called **binary exponential backoff**) after the nth collision: wait a random number of slot times from [0, 2■ – 1]. Adapts naturally to load — the more collisions, the longer the average backoff.

> **Important:** Modern switched full-duplex Ethernet has NO collisions — each device has its own dedicated link to the switch. CSMA/CD lives on only in the standard for backward compatibility. Full-duplex Ethernet just transmits whenever it wants.

CSMA/CA — CSMA with Collision Avoidance

Wireless can't detect collisions the way wired can — a receiver isn't hearing your own signal to compare. So Wi-Fi uses **collision avoidance** instead.

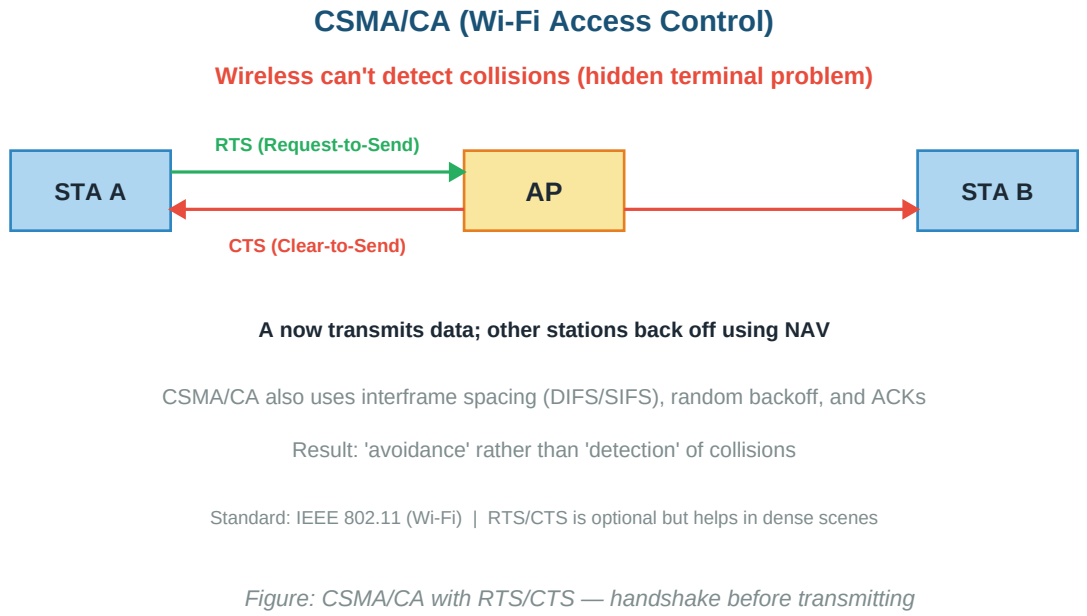


Figure: CSMA/CA with RTS/CTS — handshake before transmitting

Techniques in CSMA/CA:

- **Interframe spacing:** Wait a specific gap after sensing idle before transmitting. Higher-priority traffic uses shorter gaps (SIFS vs DIFS).
- **Random backoff:** Even after the gap, wait a random number of slots to reduce sync-triggered collisions.
- **RTS/CTS handshake:** Optional. Sender first transmits a small **Request-to-Send**; access point replies with **Clear-to-Send**; other stations hear the CTS and stay quiet. Solves the hidden terminal problem.
- **ACK for every frame:** Since we can't detect collisions, we require an ACK. No ACK → assume lost → retransmit.

4.5 Ethernet

Ethernet is the dominant LAN technology — invented at Xerox PARC in the 1970s, standardized as IEEE 802.3, still the foundation of virtually every wired network. It has evolved dramatically:

Standard	Speed	Medium
10BASE-5 / 10BASE-2	10 Mbps	Coaxial (bus)
10BASE-T	10 Mbps	Twisted pair (star)
100BASE-TX (Fast)	100 Mbps	Twisted pair
1000BASE-T (Gigabit)	1 Gbps	Twisted pair (Cat5e+)
10GBASE-T	10 Gbps	Cat6a/Cat7
10GBASE-SR/LR	10 Gbps	Fiber
40/100/400 GbE	40-400 Gbps	Fiber, data centers

Ethernet Frame Format

```
| Preamble (7B) | SFD (1B) | Dest MAC (6B) | Src MAC (6B) |
| EtherType/Length (2B) | Payload (46-1500B) | FCS/CRC (4B) |
```

The **MAC address** is a 48-bit hardware identifier hard-coded into every network card, unique worldwide. Written as six hex bytes: `aa:bb:cc:dd:ee:ff`. First three bytes = manufacturer OUI, last three = card serial.

Hubs vs Switches vs Bridges

- **Hub (L1):** Dumb signal repeater. Forwards every bit to every port. One collision domain. Half-duplex. Nearly extinct.
- **Bridge (L2):** Divides collision domains. Learns MAC addresses and forwards only where needed. Precursor to switches.
- **Switch (L2):** Multi-port bridge. Each port is its own collision domain. Learns MACs, forwards frames to the specific port where the destination lives. Enables full-duplex operation. The default device in modern LANs.

Chapter 5

Network Layer

The data link layer moves frames between neighbors. But how does data get across the world, through dozens of routers on unknown paths? That's the job of the network layer — addressing and routing. This is where IP lives.

5.1 What the Network Layer Does

The network layer's job: get a packet from a source host to a destination host, possibly across many intermediate networks. Two core problems it solves are **addressing** (uniquely identifying every host globally) and **routing** (choosing a path).

The protocol that dominates the network layer is **IP (Internet Protocol)**. IPv4 came first (1983); IPv6 is its successor (2000s) rolling out gradually. Every packet on the internet has an IP header.

5.2 IPv4 Addressing

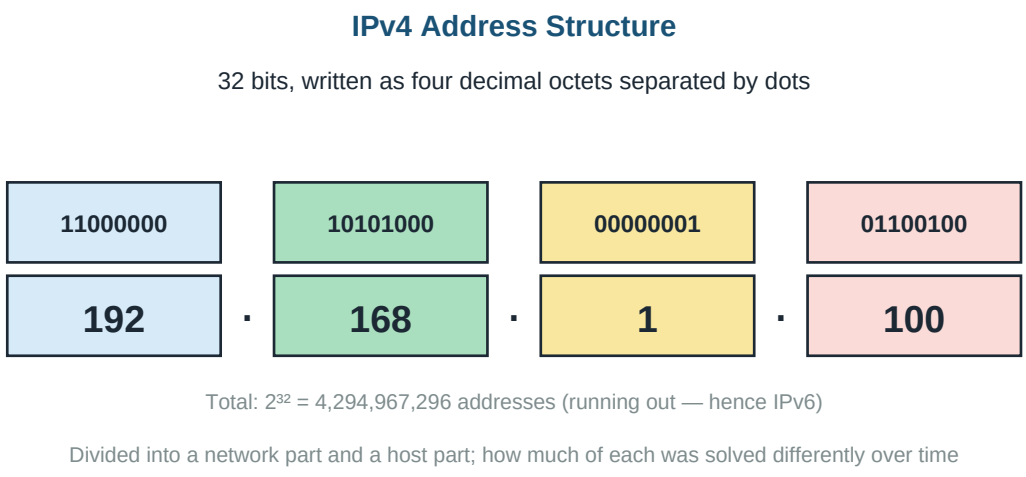


Figure: IPv4 — 32 bits written as four decimal octets

An IPv4 address is a **32-bit** number, conventionally written as four decimal octets separated by dots: `192.168.1.100`. Each octet is one byte (0-255). Total addresses: $2^{32} \approx 4.3$ billion — a number that seemed vast in 1980 and is now insufficient.

Network and Host Parts

An IP address has two parts: a **network portion** (identifies which network the host is on) and a **host portion** (identifies the specific host within that network). Routers care about the network part; the host part matters only at the final destination network.

Classful Addressing

The original scheme (1981) divided the address space into five classes based on the first few bits, each with a fixed network/host split.

Classful IPv4 Addressing

A	0	1-126	Very large orgs	/8	8
B	10	128-191	Medium orgs	/16	16
C	110	192-223	Small orgs (LANs)	/24	24
D	1110	224-239	Multicast	-	-
E	1111	240-255	Reserved (research)	-	-

127.x.x.x is loopback (localhost) — reserved outside classes

Classful addressing wasted addresses → replaced by CIDR (classless)

Private IPs (RFC 1918): 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

Figure: Classful IPv4 — five classes A through E

Important: Classful addressing wasted addresses on a massive scale. A university needing 500 addresses would be given a Class B (65,534 addresses) — wasting 65,000. This is one reason IPv4 exhaustion happened so much faster than expected. Classful addressing was replaced by CIDR in 1993.

Classless Inter-Domain Routing (CIDR)

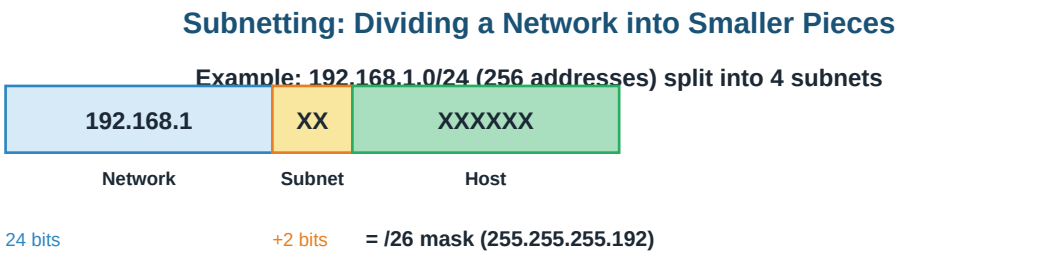
CIDR uses **variable-length prefixes**. Instead of fixed 8/16/24-bit boundaries, the network portion can be any length. Notation: `192.168.1.0/24` means the first 24 bits are the network, leaving 8 for hosts.

```
192.168.1.0/24 → subnet mask 255.255.255.0 → 256 addresses (254 usable)
10.0.0.0/16 → subnet mask 255.255.0.0 → 65,536 addresses
172.16.0.0/12 → subnet mask 255.240.0.0 → 1,048,576 addresses
```

CIDR enabled ISPs to allocate blocks of exactly the size needed, dramatically slowing IPv4 exhaustion. It also allows **route aggregation** (multiple small routes combined into one larger one) which keeps routing tables manageable.

5.3 Subnetting

Subnetting divides one large network into several smaller ones. You "borrow" some bits from the host portion to become subnet bits. The advantages are smaller broadcast domains, better security (traffic can be filtered between subnets), and organizational alignment (each department gets its own subnet).



Subnet 1	192.168.1.0/26	hosts .0 - 63
Subnet 2	192.168.1.64/26	hosts .64 - 127
Subnet 3	192.168.1.128/26	hosts .128 - 191
Subnet 4	192.168.1.192/26	hosts .192 - 255

Each subnet: 64 addresses; 62 usable (subtract network + broadcast)

Figure: Subnetting — borrow bits from host portion to create sub-networks

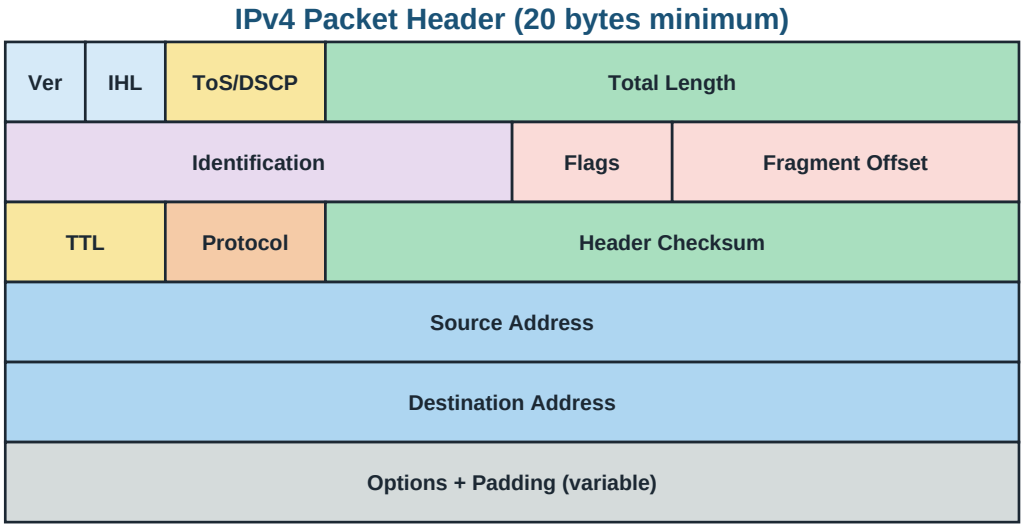
Special Addresses in Every Subnet

- **Network address** (host bits all 0): identifies the subnet itself. Not assigned to any host. E.g., 192.168.1.0/24 → 192.168.1.0.
- **Broadcast address** (host bits all 1): sends to every host in the subnet. E.g., 192.168.1.0/24 → 192.168.1.255.
- So a /24 subnet has 256 addresses total but only 254 usable hosts.

Special IPv4 Address Ranges

Range	Meaning
10.0.0.0/8	Private (RFC 1918)
172.16.0.0/12	Private (RFC 1918)
192.168.0.0/16	Private (RFC 1918)
127.0.0.0/8	Loopback (localhost)
169.254.0.0/16	Link-local (APIPA)
224.0.0.0/4	Multicast (Class D)
0.0.0.0/8	"This host" (used during boot)
255.255.255.255	Limited broadcast

5.4 IPv4 Packet Header



Key fields: TTL (hop limit), Protocol (TCP=6, UDP=17, ICMP=1), Src/Dst addresses

Header checksum covers only header, not payload — recomputed at every router

Figure: IPv4 header — 20 bytes minimum, up to 60 with options

Key fields:

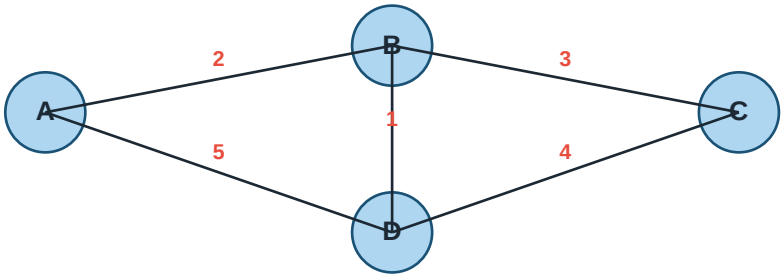
- **Version (4 bits):** IP version. 4 for IPv4, 6 for IPv6.
- **IHL (4 bits):** Header length in 32-bit words. Minimum 5 (= 20 bytes).
- **Total Length (16 bits):** Entire packet length in bytes — maximum 65,535.
- **Identification, Flags, Fragment Offset:** Used for fragmentation. If a packet is too big for the next link's MTU, it gets fragmented and these fields help reassemble.
- **TTL (Time to Live, 8 bits):** Decremented by every router. When 0, packet is discarded. Prevents packets from circulating forever in routing loops. Typically starts at 64 or 128.
- **Protocol (8 bits):** What's inside — TCP=6, UDP=17, ICMP=1.
- **Header Checksum:** Covers only the header. Recomputed at every router (since TTL changes).
- **Source/Destination Address:** 32-bit IP addresses.

5.5 Routing

The internet is a network of networks — millions of routers. When a packet arrives, the router looks at the destination IP and decides which outgoing link to send it on. That decision comes from a **routing table**, built by **routing protocols**.

Distance-Vector Routing

Distance-Vector Routing (Bellman-Ford)



Each router keeps a table:

[dest, next hop, cost]

Periodically shares vector with neighbors

Uses Bellman-Ford update: my_cost[dest] = min over neighbors of **(cost_to_neighbor + neighbor's cost_to_dest)**

Issue: count-to-infinity on failure. Fix: split horizon, poison reverse.

Figure: Distance vector — routers share cost vectors with neighbors

Each router maintains a table of (destination, next hop, cost). Periodically it sends its distance vector to each neighbor. When a router receives a neighbor's vector, it updates its own using the **Bellman-Ford** equation:

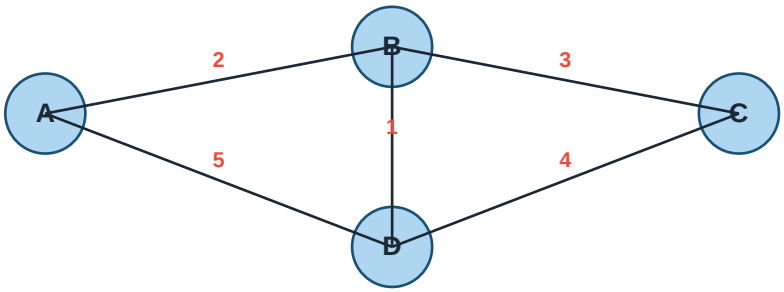
```
cost_to(D) = min over all neighbors N of:
cost_to(N) + N's_cost_to(D)
```

Important: Distance-vector suffers from the **count-to-infinity** problem. When a link fails, routers may "agree" that the failed destination is reachable via each other, incrementing costs slowly. Mitigations: **split horizon** (don't advertise a route back the way you learned it), **poison reverse** (actively advertise infinite cost back).

RIP (Routing Information Protocol) is the classic distance-vector protocol. Uses hop count as cost, max 15 hops (16 = unreachable). Simple, but slow to converge.

Link-State Routing

Link-State Routing (Dijkstra's algorithm)



1. Each router floods link-state info to ALL others
2. Every router builds a COMPLETE topology map
3. Each router runs Dijkstra from ITSELF to find shortest paths
4. Faster convergence than distance-vector, uses more memory

Example protocol: OSPF (Open Shortest Path First)

Figure: Link state — every router builds a complete topology map

Every router discovers its own links and their costs, then **floods** this info to every other router. Each router builds a complete map of the network. Then each runs **Dijkstra's algorithm** from itself to compute shortest paths to every destination.

Advantages: fast convergence, no count-to-infinity, better handling of large networks. Disadvantage: requires more memory and CPU. **OSPF (Open Shortest Path First)** is the standard link-state protocol, used in most enterprise networks.

Path-Vector Routing

Used between **autonomous systems (ASes)** — different organizations' networks. Advertises entire paths, not just distances. Enables policy-based routing: an AS can prefer some paths over others for business reasons. **BGP (Border Gateway Protocol)** is the path-vector protocol that stitches the internet together.

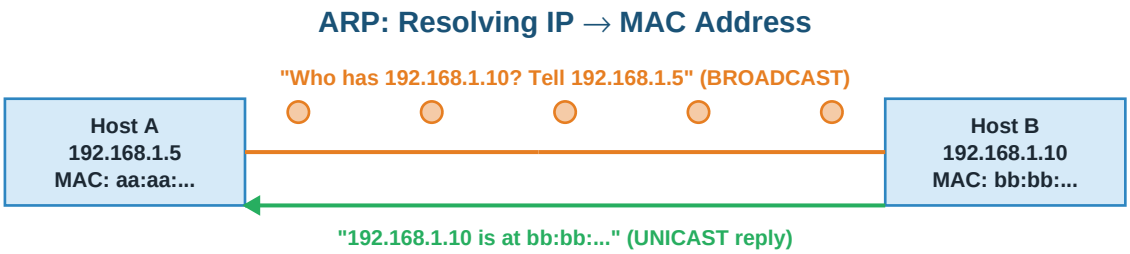
Interior vs Exterior Gateway Protocols

- **Interior (IGP):** Within one AS. Examples: RIP, OSPF, EIGRP. Optimize for shortest path.
- **Exterior (EGP):** Between ASes. Only BGP. Optimizes for policy compliance, not shortest path.

5.6 Supporting Protocols

ARP — Address Resolution Protocol

IP addresses are logical; MAC addresses are physical. When a host wants to send an IP packet to another host on the same LAN, it needs the destination's MAC. **ARP** resolves IP → MAC.



ARP maps L3 IP → L2 MAC. Result cached in ARP table.

RARP does the reverse (MAC → IP), but DHCP has mostly replaced it.

IPv6 uses NDP (Neighbor Discovery Protocol) instead of ARP.

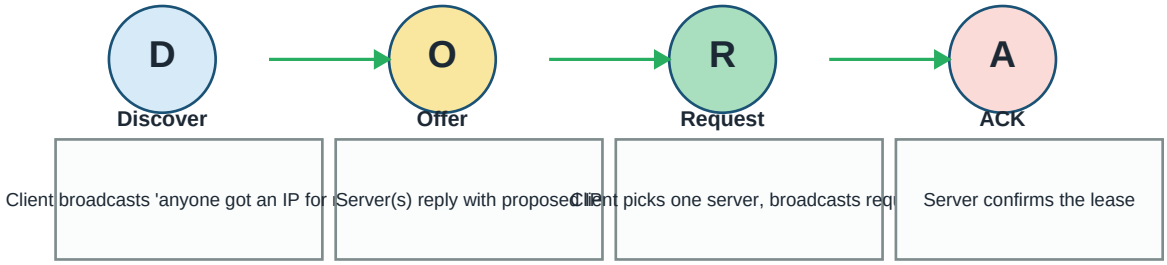
Figure: ARP — broadcast the question, unicast the answer

The host broadcasts "Who has IP 192.168.1.10?" Every host on the LAN receives it, but only 192.168.1.10 replies with its MAC. The result is cached in an **ARP table** to avoid repeat queries.

DHCP — Dynamic Host Configuration Protocol

Manually configuring IPs on every host doesn't scale. DHCP automates it. When a new host joins, it gets an IP address (and DNS servers, default gateway, subnet mask) from a DHCP server. The exchange follows a four-step process nicknamed **DORA**.

DHCP: Automatic IP Configuration (DORA)



Server also tells client: subnet mask, default gateway, DNS servers, lease time

Uses UDP; server port 67, client port 68

Figure: DHCP DORA — Discover, Offer, Request, Acknowledge

ICMP — Internet Control Message Protocol

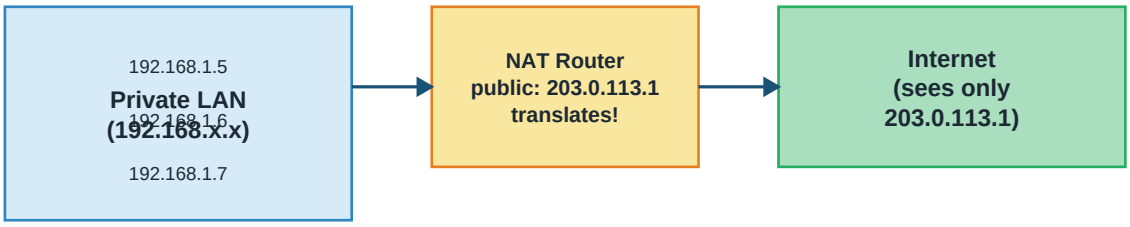
The signaling protocol of IP. Used for error reporting and diagnostics — not for user data. Two familiar tools use ICMP heavily:

- **ping:** sends ICMP Echo Request, gets Echo Reply. Measures reachability and round-trip time.
- **traceroute:** Sends packets with TTL=1, 2, 3... Each expired-TTL router replies with ICMP Time Exceeded. From these replies, we discover every router along the path.

NAT — Network Address Translation

Private IPv4 addresses (10.x, 172.16-31.x, 192.168.x) aren't routable on the public internet. NAT translates between private and public addresses at the network boundary — usually your home router.

NAT: Making Many Private IPs Look Like One Public IP



NAT translation table:

192.168.1.5:5000 ↔ 203.0.113.1:6000
192.168.1.6:5001 ↔ 203.0.113.1:6001
192.168.1.7:5002 ↔ 203.0.113.1:6002

Different (private IP, port) pairs mapped to different (public IP, port) pairs

Helped delay IPv4 exhaustion; breaks true end-to-end connectivity

Figure: NAT — many private addresses share one public address

The specific flavor typically used at home is **PAT (Port Address Translation)** or **NAPT**: translation uses BOTH IP and port so many private hosts can share one public IP. Each active flow gets a unique (public IP, port) pair.

> **Important:** NAT breaks the internet's end-to-end principle. A random host on the internet can't initiate a connection to a host behind NAT — the NAT box wouldn't know which private host to forward to. This is why P2P applications need workarounds like STUN, TURN, and hole-punching.

5.7 IPv6 — The Long Migration

IPv4's 32-bit address space (4.3 billion) has been depleted. **IPv6** uses **128 bits** — that's $2^{128} \approx 3.4 \times 10^{38}$ addresses, enough to give every atom on Earth's surface many addresses.

Notation: eight groups of four hex digits separated by colons:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
Shortened: leading zeros dropped, longest run of zeros → '::'
2001:db8:85a3::8a2e:370:7334
```

Improvements over IPv4

- **Vast address space** — no exhaustion in any foreseeable future.
- **Simpler header** — fixed 40-byte size; extensions are separate.
- **No fragmentation at routers** — sender's job to size correctly (uses Path MTU Discovery).
- **Built-in security (IPsec)** — encryption/authentication as standard, not add-on.
- **Better multicast** — first-class support, no broadcast (uses multicast instead).
- **Autoconfiguration** — hosts can generate their own addresses (SLAAC) without DHCP.

Adoption has been slow because IPv4 with NAT is "good enough" for most users. But major services, mobile networks, and cloud platforms are increasingly IPv6-first.

Chapter 6

Transport Layer

The network layer gets packets from host to host. But hosts run multiple applications. Which app is the packet for? And is IP reliable enough? This is where the transport layer comes in — with TCP for reliability and UDP for speed.

6.1 Ports and Sockets

The network layer identifies hosts by IP address. The transport layer identifies applications on a host by **port number**. IP + port together — a **socket** — uniquely identify one endpoint of a communication.

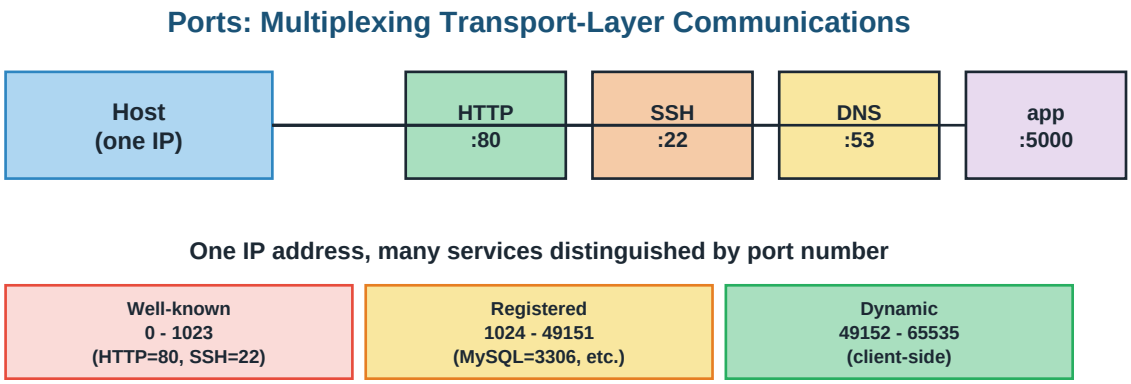


Figure: Ports — one IP, many services distinguished by port number

Port numbers are 16 bits (0-65535), divided into three ranges:

- **Well-known ports (0-1023):** Reserved for standard services. HTTP = 80, HTTPS = 443, SSH = 22, FTP = 21, SMTP = 25, DNS = 53. On Unix systems, binding to these requires root privileges.
- **Registered ports (1024-49151):** Assigned by IANA to specific applications. MySQL = 3306, PostgreSQL = 5432, Redis = 6379.
- **Dynamic/private ports (49152-65535):** Not registered. Clients use these as their source port for outgoing connections.

Key Insight: A TCP connection is uniquely identified by the 4-tuple **(src IP, src port, dst IP, dst port)**. One server on port 80 can serve millions of clients because each client connection has a different source port.

6.2 UDP — Fast and Loose

UDP (User Datagram Protocol) is the minimal transport layer — barely more than a wrapper around IP. No connection setup, no ordering, no retransmission, no flow control. If a packet is lost, it's lost. That sounds bad, but for many use cases it's exactly right.

UDP Header (8 bytes only)

Fire-and-forget: no connection, no sequencing, no retransmission, no flow control

Perfect for: DNS, video streaming, gaming, VoIP — small overhead, real-time

Figure: UDP header — just 8 bytes

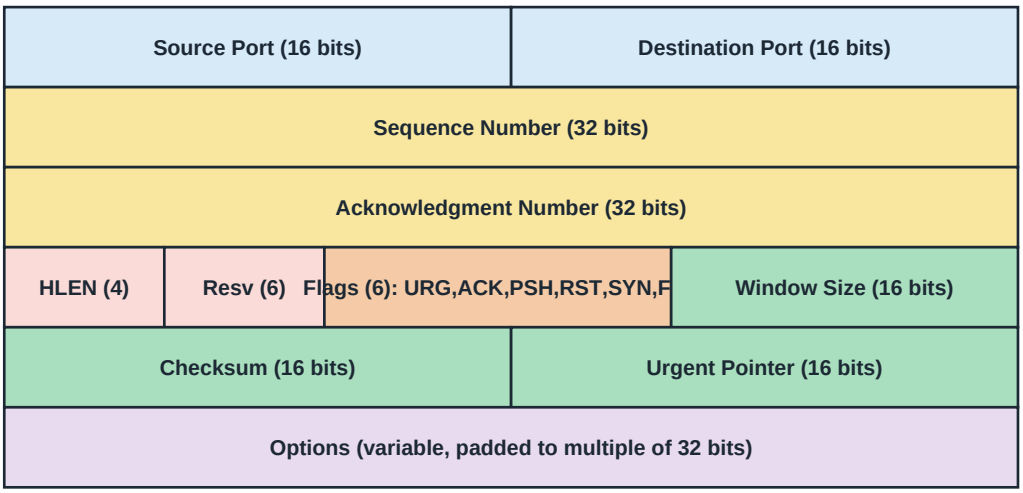
When to Use UDP

- **Real-time media (voice, video):** A retransmitted audio packet arrives too late to be useful. Better to drop it and keep going.
- **DNS queries:** Short request, short response. Setting up a TCP connection would add more time than sending the whole thing over UDP.
- **Gaming:** Latency matters more than perfect delivery. Modern positions matter more than history.
- **DHCP:** A client that doesn't have an IP yet can't set up a TCP connection.
- **SNMP, TFTP, NTP:** Small, simple protocols where reliability is handled by the application.

6.3 TCP — Reliable and Ordered

TCP (Transmission Control Protocol) is the workhorse of the internet. It gives applications a reliable, ordered, in-order byte stream — as if two applications had a private wire between them. Behind the scenes, it does an enormous amount of work: connection setup, sequence numbering, acknowledgments, retransmissions, flow control, and congestion control.

TCP Header (20 bytes minimum)



Rich header enables reliability, ordering, flow control, congestion control

Figure: TCP header — 20 bytes minimum, packed with control fields

Key TCP Header Fields

- **Sequence Number (32 bits):** Byte offset of the first byte in this segment within the whole data stream. Random initial value (ISN) for security.
- **Acknowledgment Number:** Next byte the sender expects to receive. Cumulative ACK.
- **Window Size:** How many more bytes the receiver is willing to accept. Used for flow control.
- **Flags:** SYN (synchronize), ACK (acknowledge), FIN (finish), RST (reset), PSH (push), URG (urgent).

6.4 TCP Connection Establishment

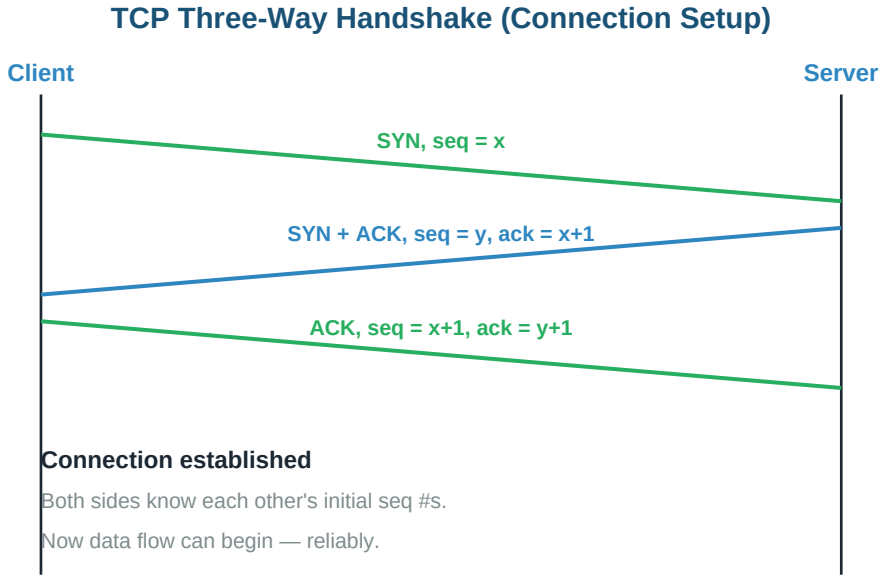


Figure: Three-way handshake — SYN, SYN-ACK, ACK

Before any data flows, TCP establishes a connection with a **three-way handshake**:

- **Step 1:** Client sends `SYN, seq=x`. Client is trying to open.
- **Step 2:** Server responds with `SYN + ACK, seq=y, ack=x+1`. Server agrees, and picks its own initial sequence number y.
- **Step 3:** Client sends `ACK, seq=x+1, ack=y+1`. Client acknowledges.

After this, both sides know each other's initial sequence numbers. The connection is established, and data can flow in both directions.

Important: The handshake exposes TCP to **SYN flood attacks**: an attacker sends many SYNs but never completes the handshake. The server allocates memory for each pending connection until it runs out. Modern defenses use **SYN cookies** — the server doesn't allocate state until the final ACK confirms.

Connection Termination

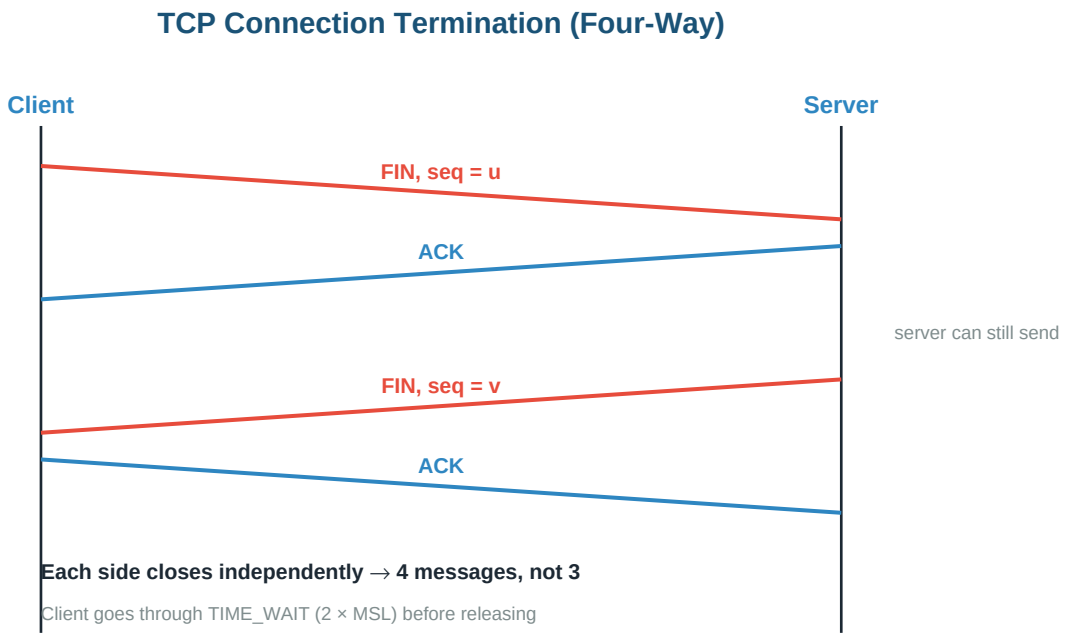


Figure: TCP connection close — four-way, each side closes independently

Closing a TCP connection takes **four** messages, not three, because each side closes independently. One side sends FIN when done sending; the other acknowledges but may still have data to send. When it's also done, it sends its own FIN. After the last ACK, the initiator waits **TIME_WAIT** (2 × MSL, about 2 minutes) before releasing resources — this handles any lingering duplicate packets.

6.5 Reliability, Flow Control, Congestion Control

Reliability

Every byte gets a sequence number. Receiver ACKs by naming the next byte it expects (cumulative ACK). If sender doesn't get ACKs within an RTO (Retransmission Timeout), it retransmits. RTO is dynamically estimated from observed round-trip times using EWMA smoothing.

Flow Control — Don't Overwhelm the Receiver

The receiver advertises a **receive window** in every ACK: "I can accept this many more bytes." The sender never has more unacknowledged data outstanding than the advertised window. This prevents a fast sender from flooding a slow receiver.

Congestion Control — Don't Overwhelm the Network

Flow control protects the endpoints. Congestion control protects the network — routers dropping packets because their buffers are full. TCP maintains its own **congestion window (cwnd)**. The effective sending window is min(receive_window, cwnd).

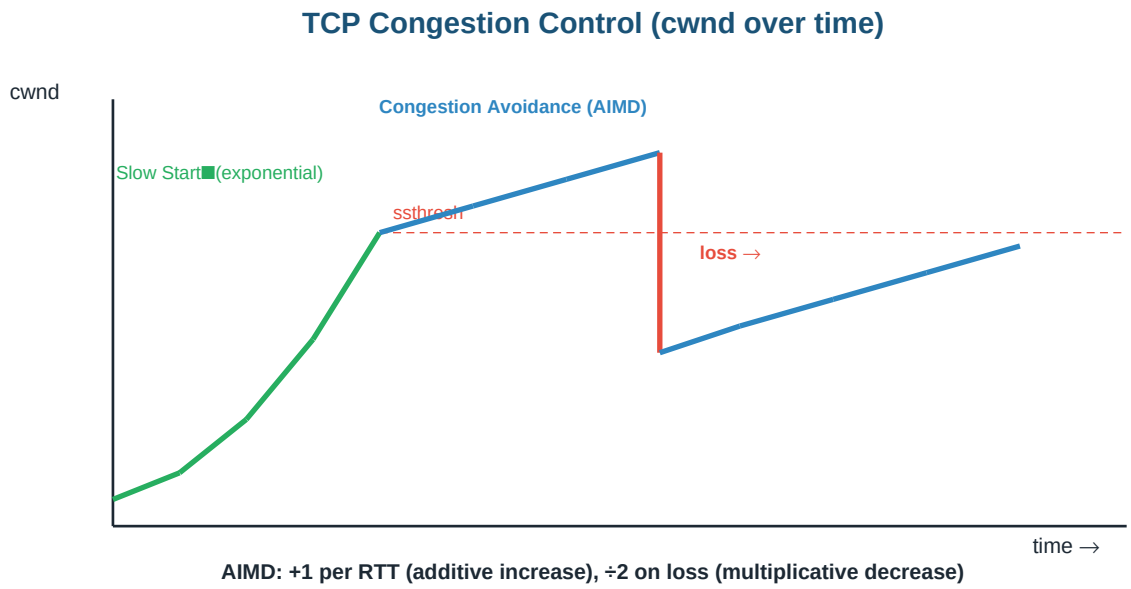


Figure: TCP congestion window evolves through slow start, congestion avoidance, and recovery

Slow Start

Start conservatively: cwnd = 1 MSS. For each ACK received, cwnd += 1 MSS. This doubles cwnd every RTT — exponential growth. Continues until cwnd reaches **ssthresh** (slow-start threshold) or a loss is detected.

Congestion Avoidance

Once cwnd ≥ ssthresh, grow linearly instead: cwnd += 1 MSS per RTT. This is called **AIMD (Additive Increase, Multiplicative Decrease)** — the additive increase side.

On Loss Detection

- **Timeout (severe loss):** Halve ssthresh, set cwnd = 1 MSS, restart with slow start. This is Tahoe's behavior.
- **Three duplicate ACKs (mild loss):** Halve ssthresh, set cwnd = new ssthresh, do fast retransmit and continue with congestion avoidance. This is Reno's **fast recovery**.

Key Insight: AIMD makes TCP **fair**: multiple flows sharing a bottleneck converge to roughly equal rates. It's also why TCP is called "self-clocking" — the pace of ACKs sets the pace of sending.

6.6 TCP vs UDP — When to Use Which

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable (retransmission)	Best effort
Ordering	Preserved	Not guaranteed
Flow control	Yes (window)	No
Congestion control	Yes (cwnd)	No
Header size	20+ bytes	8 bytes

Feature	TCP	UDP
Speed	Slower (overhead)	Faster (minimal overhead)
Use cases	HTTP, SSH, email, file transfer	DNS, VoIP, gaming, streaming

Chapter 7

Application Layer

This is the layer users see. Web browsing, email, DNS lookups, remote logins — all live here. This chapter tours the most important application-layer protocols that make the internet actually useful.

7.1 The Client-Server Paradigm

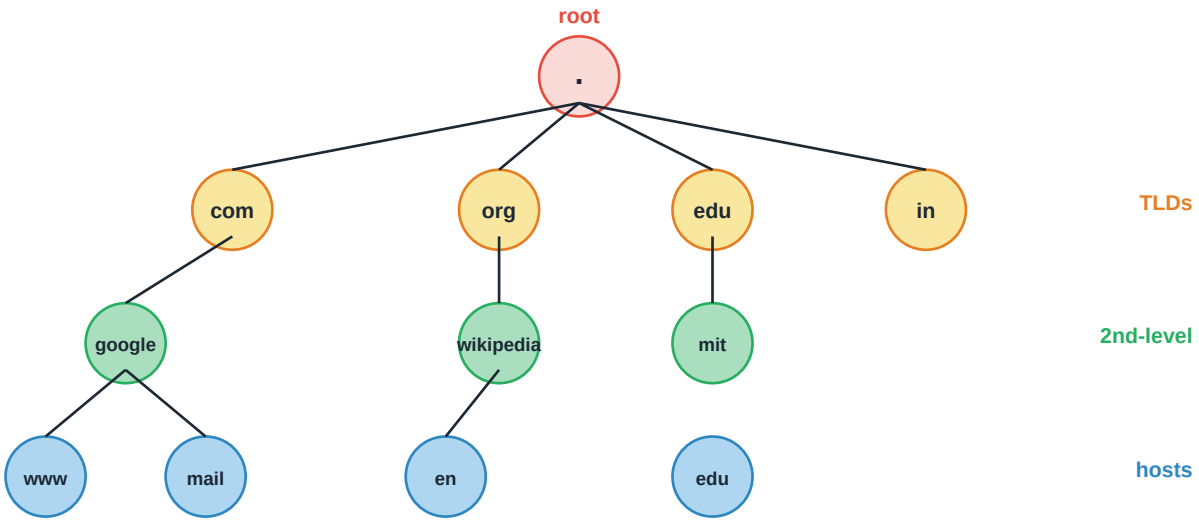
Most application protocols follow a **client-server** model. A client initiates a request; a server processes it and responds. The server is always ready, waiting on a well-known port. The client comes and goes.

Some applications (BitTorrent, blockchain networks, some messaging apps) use **peer-to-peer (P2P)** instead — every participant is both client and server. P2P scales differently: the more users, the more capacity.

7.2 DNS — The Internet's Phone Book

Humans remember `google.com`. Computers need `142.250.190.14`. **DNS (Domain Name System)** translates one to the other. Every time you type a URL, click a link, or send an email, DNS is quietly at work.

DNS: Hierarchical Namespace



Read RIGHT to LEFT: www.google.com. → find 'com' in root, 'google' in com, 'www' in google

Figure: DNS namespace — hierarchical from root down

DNS Hierarchy

The DNS namespace is a tree. At the root is **"." (dot)** — usually implicit. Below the root are **Top-Level Domains (TLDs)**: generic (com, org, net, edu) and country-code (in, uk, jp). Below TLDs are registered domains (google.com), then subdomains (mail.google.com).

The tree is hosted by many DNS servers, each responsible for a portion of the namespace. No single server knows everything.

DNS Resolution

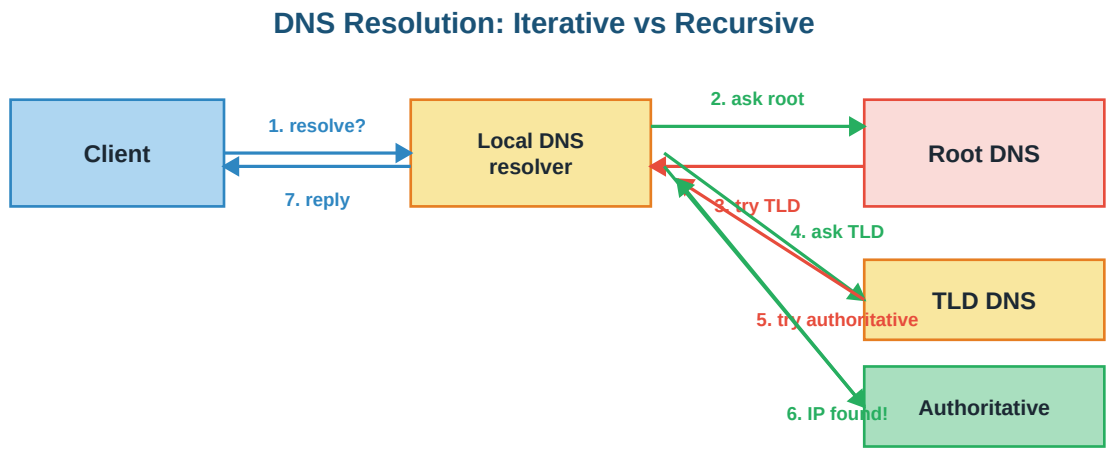


Figure: DNS resolution — recursive from client, iterative to authoritative servers

The typical resolution flow:

- **1.** Your computer asks its local DNS resolver (usually your ISP's or a public one like 8.8.8.8) — "What's the IP for www.example.com?" This is a **recursive** query — the resolver must return a full answer.
- **2.** The resolver checks its cache. If not found, it asks a **root DNS server** — which doesn't know the answer but points to the .com TLD server.
- **3.** The resolver asks the **.com TLD server**, which points to the authoritative server for example.com.
- **4.** The resolver asks the **authoritative server for example.com**, which returns the IP for www.
- **5.** The resolver caches the answer and returns it to your computer. Steps 2-4 are **iterative** — each server just refers to the next.

DNS Record Types

- **A:** Maps a name to an IPv4 address.
- **AAAA:** Maps a name to an IPv6 address.
- **CNAME:** Canonical name — an alias for another name.
- **MX:** Mail exchanger — server accepting email for this domain.
- **NS:** Name server — which DNS server is authoritative for this zone.
- **TXT:** Free-form text; used for verification, SPF, DKIM.
- **PTR:** Reverse lookup — IP to name.

DNS uses UDP on port 53 for most queries (small, fast). Larger responses or zone transfers use TCP. Modern DNS security additions: DNSSEC (signed records), DoH/DoT (DNS over HTTPS/TLS for privacy).

7.3 HTTP — The Web

HTTP (Hypertext Transfer Protocol) is the language of the web. Every time your browser requests a page, image, or API call, it uses HTTP. It's a stateless request-response protocol on top of TCP (port 80) or TLS (port 443, called HTTPS).

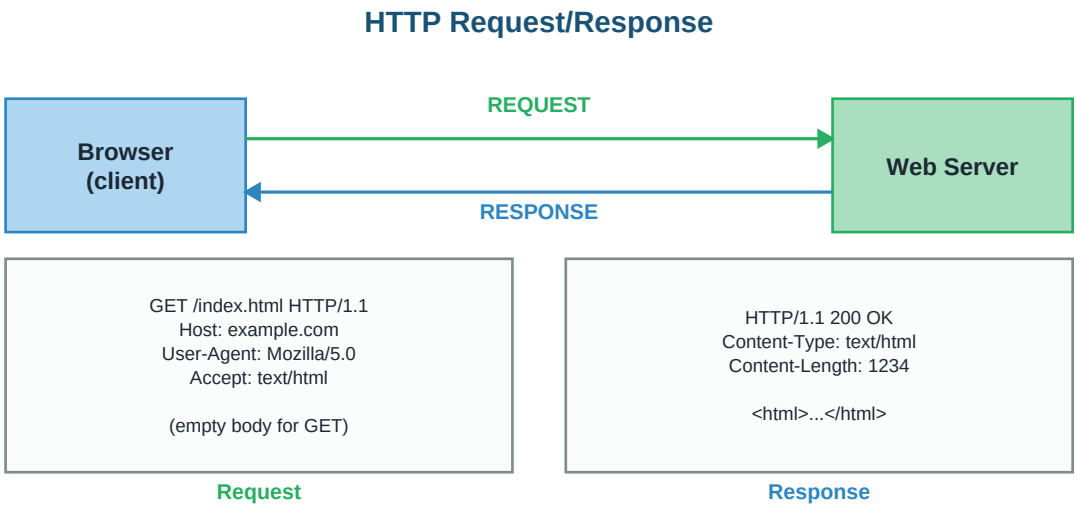


Figure: HTTP request and response — headers and body

HTTP Methods

- **GET:** Retrieve a resource. Safe (no side effects), idempotent.
- **POST:** Submit data. Not safe, not idempotent.
- **PUT:** Create or replace a resource. Idempotent.
- **DELETE:** Remove a resource. Idempotent.
- **PATCH:** Partial update to a resource.
- **HEAD:** Like GET but no body — just headers.
- **OPTIONS:** Ask what methods are allowed. Used in CORS.

HTTP Status Codes

- **1xx Informational:** 100 Continue.
- **2xx Success:** 200 OK, 201 Created, 204 No Content.
- **3xx Redirection:** 301 Moved Permanently, 304 Not Modified.
- **4xx Client Error:** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests.
- **5xx Server Error:** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout.

HTTP Evolution

- **HTTP/1.0:** One TCP connection per request. Wasteful — each request pays connection setup cost.
- **HTTP/1.1:** Persistent connections (keep-alive). Multiple requests over one TCP connection. Pipelining (send multiple requests without waiting). Chunked transfer encoding.

- **HTTP/2 (2015):** Binary framing, multiplexed streams over one connection, header compression (HPACK), server push.
- **HTTP/3 (2022):** Runs over QUIC (UDP-based) instead of TCP. Avoids TCP's head-of-line blocking, faster connection setup, better on mobile.

7.4 Email — SMTP, POP3, IMAP

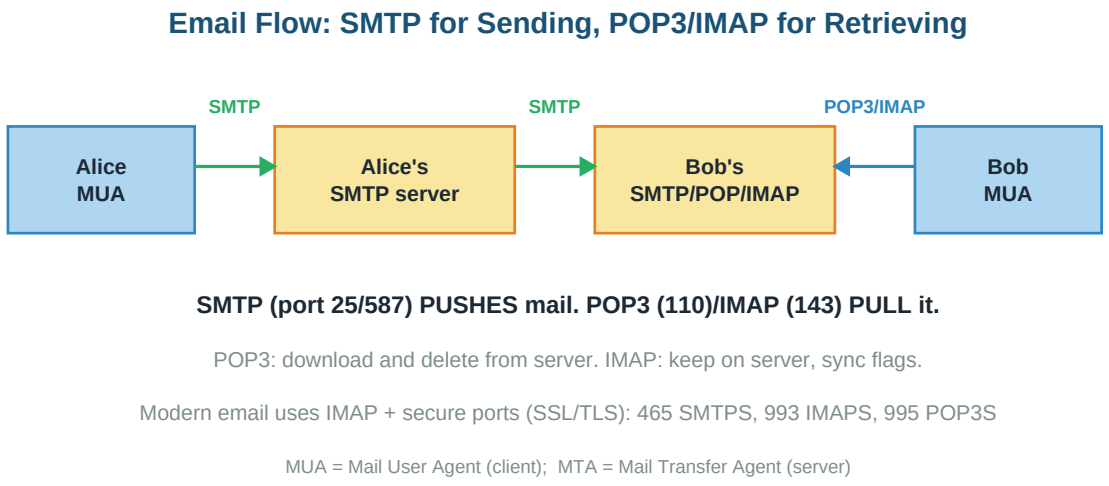


Figure: Email uses different protocols for sending vs receiving

SMTP — Sending Mail

SMTP (Simple Mail Transfer Protocol) handles sending mail. When you click send in your email client, it uses SMTP to hand the message to your outbound mail server. That server then uses SMTP again to relay the message to the recipient's mail server. Standard port: 25 (server-to-server), 587 (client-to-server with auth), 465 (SMTPS with TLS).

POP3 — Simple Mail Retrieval

POP3 (Post Office Protocol version 3): mail client connects to the mail server, downloads all messages, deletes them from the server. Simple. Great for a single device — but you can't check mail from your laptop and see the same state on your phone. Port 110 (or 995 for POP3S).

IMAP — Multi-Device Mail

IMAP (Internet Message Access Protocol): mail stays on the server. Clients view and manage it remotely. Read/unread flags, folder structure, and message state sync across all your devices. Modern email uses IMAP. Port 143 (or 993 for IMAPS).

7.5 FTP, TELNET, SSH

FTP — File Transfer Protocol

One of the oldest internet protocols. Uses **two connections**: a **control connection** (port 21) for commands, and a **data connection** (port 20, active mode) for the actual file bytes. Two modes: **active** (server connects back to client for data) and **passive** (client initiates both, works better through NAT/firewalls).

> **Important:** FTP sends passwords in cleartext. Use **SFTP** (SSH File Transfer Protocol) or **FTPS** (FTP over TLS) for security.

TELNET

Remote terminal access. Interactive login to a remote machine. Extremely simple protocol — but sends everything, including passwords, in cleartext. Considered obsolete for security. Still used for diagnosing text-based services ("can I telnet to port 80?").

SSH — Secure Shell

The modern replacement for TELNET. Provides encrypted remote login and secure command execution. Supports public-key authentication (much stronger than passwords), port forwarding, X11 forwarding, and file transfer (SCP, SFTP). Port 22. If you administer a Linux server, you use SSH constantly.

Chapter 8

Network Security

Networks carry sensitive data — passwords, financial info, private conversations. Attackers are everywhere. Security isn't optional. This chapter covers the cryptographic building blocks and practical defenses that keep networks trustworthy.

8.1 What Are We Protecting?

Network security aims at three fundamental properties, often called the **CIA triad**:

- **Confidentiality:** Only authorized parties can read the data. Prevents eavesdropping.
- **Integrity:** The data hasn't been altered in transit. Prevents tampering.
- **Availability:** The service is accessible when needed. Prevents denial of service.

Two additional properties are often added:

- **Authentication:** Verifying who someone is.
- **Non-repudiation:** The sender can't later deny sending the message.

Common Threats

- **Eavesdropping:** Passive attacker reads traffic on the wire. Defense: encryption.
- **Man-in-the-Middle (MITM):** Attacker relays messages between two parties, unbeknownst to them. Defense: mutual authentication + encryption.
- **Replay attack:** Attacker captures legitimate messages and replays them later. Defense: nonces, timestamps, sequence numbers.
- **Denial of Service (DoS/DDoS):** Attacker floods a service, making it unavailable to real users. Defense: rate limiting, upstream filtering, over-provisioning.
- **DNS/ARP spoofing:** Attacker forges directory responses to redirect traffic. Defense: DNSSEC, static ARP entries, network monitoring.
- **Phishing:** Social engineering to trick users into revealing credentials. Defense: user education, 2FA.

8.2 Symmetric-Key Cryptography

The oldest form of cryptography. Both parties share the same secret key. The key is used both to encrypt and to decrypt.

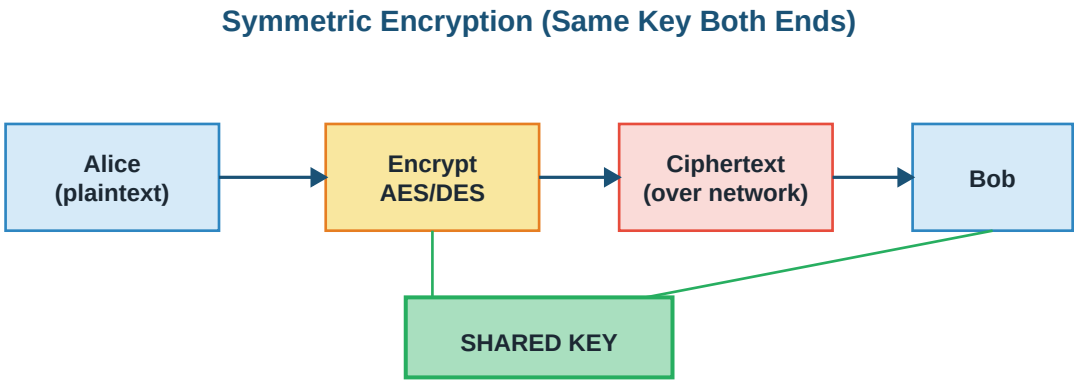


Figure: Symmetric encryption — the same key encrypts and decrypts

Modern symmetric algorithms:

- **AES (Advanced Encryption Standard):** Block cipher, 128-bit blocks, 128/192/256-bit keys. The current standard for symmetric encryption. Fast, secure, widely supported in hardware.
- **DES:** Older standard, 64-bit blocks, 56-bit key. Broken by modern hardware. Legacy only.
- **3DES (Triple DES):** Apply DES three times with different keys. Legacy — being phased out for AES.
- **ChaCha20:** Stream cipher popular in TLS, especially where AES hardware acceleration isn't available. Used with Poly1305 for authentication (ChaCha20-Poly1305).

Important: Symmetric encryption is fast but has a fundamental problem: **how do you share the key securely in the first place?** You can't send it over the same channel you're trying to protect. This is the **key distribution problem**, solved by asymmetric cryptography.

8.3 Asymmetric-Key Cryptography

Also called **public-key cryptography**. Each party has TWO mathematically-related keys: a **public key** that everyone knows, and a **private key** that only they know. Data encrypted with one key can only be decrypted with the other.

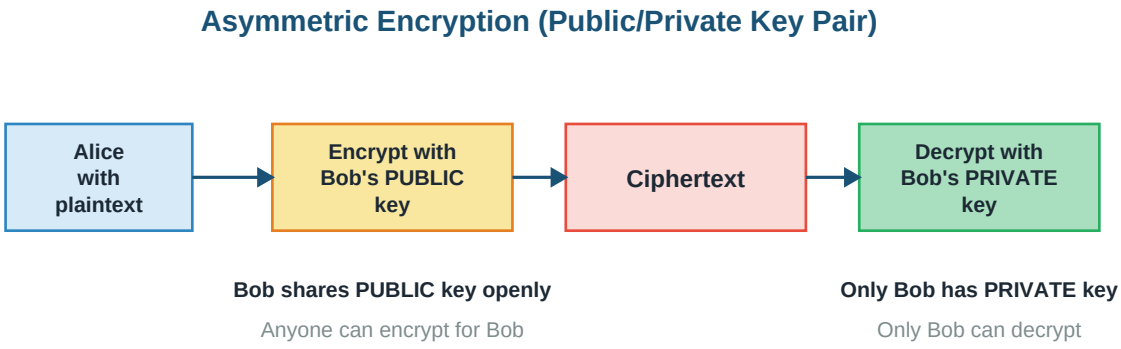


Figure: Asymmetric encryption — encrypt with public key, decrypt with private key

To send Bob a confidential message, Alice encrypts it with Bob's PUBLIC key. Since only Bob has the matching PRIVATE key, only Bob can decrypt it. Anyone can send Bob confidential messages without any prior secret exchange.

RSA — The Classic Algorithm

Named after Rivest, Shamir, Adleman (1977). Its security rests on the difficulty of factoring the product of two large primes. Key sizes typically 2048 or 4096 bits.

```
Key generation:
1. Pick two large primes p, q
2. n = p × q (this becomes public)
3. ϕ(n) = (p−1)(q−1)
4. Pick e coprime with ϕ(n); typically 65537
5. Find d such that (e × d) mod ϕ(n) = 1

Public key: (n, e) | Private key: (n, d)
Encrypt: c = m^e mod n
Decrypt: m = c^d mod n
```

ECC — Elliptic Curve Cryptography

Newer approach based on the mathematics of elliptic curves. Provides equivalent security to RSA with much shorter keys — 256-bit ECC ≈ 3072-bit RSA. Faster and less resource-hungry, especially important for mobile devices. Used in modern TLS, Bitcoin, iMessage.

Hybrid Encryption in Practice

Asymmetric encryption is slow — 100x to 1000x slower than symmetric. So real systems use **hybrid encryption**: use asymmetric to exchange a random symmetric session key, then use the symmetric key for the actual data. This is how TLS (HTTPS) works.

8.4 Hash Functions

A **cryptographic hash function** takes an input of any length and produces a fixed-length "fingerprint". Properties required:

- **Deterministic:** Same input always produces same output.
- **One-way:** Given a hash, infeasible to find the original input.
- **Avalanche effect:** A tiny change in input produces a completely different hash.
- **Collision-resistant:** Hard to find two different inputs with the same hash.

Common hash functions:

- **MD5:** Broken. Do not use for security. Still occasionally seen for checksums.
- **SHA-1:** Broken. Do not use for security. Deprecated.
- **SHA-256, SHA-512:** Current standards from the SHA-2 family. Widely used.
- **SHA-3:** Newer alternative with a different internal design. Safe for the future.

Uses of hashes: password storage (never store plaintext!), file integrity, digital signatures, blockchain.

8.5 Digital Signatures

A **digital signature** proves both that a message came from a specific sender (authentication) and that it hasn't been tampered with (integrity). The elegant trick: use asymmetric encryption in REVERSE.

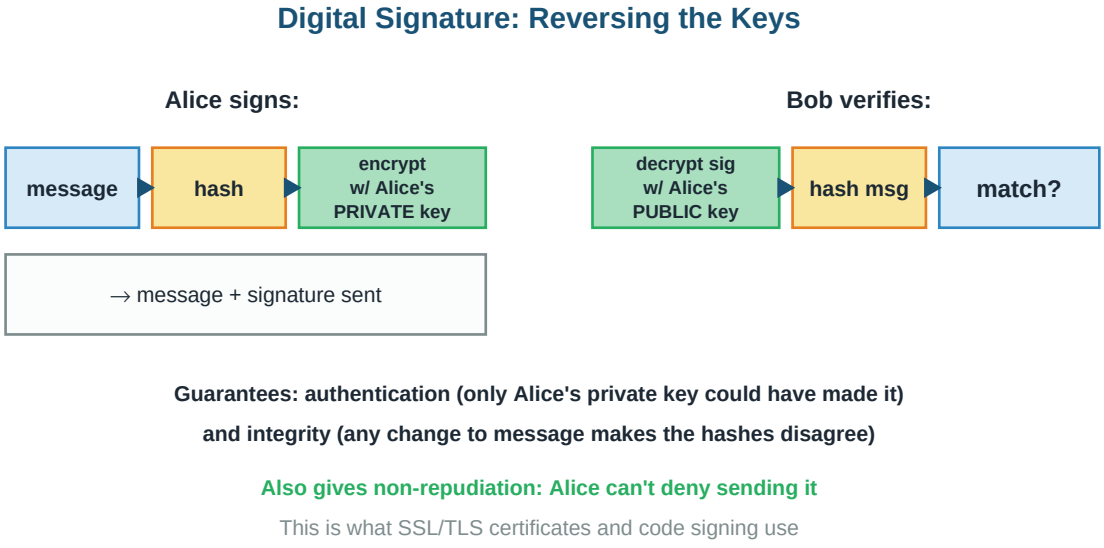


Figure: Digital signatures — encrypt with private key, verify with public key

The process:

- **Signing:** Alice hashes the message and encrypts the hash with her PRIVATE key. The encrypted hash is the signature. Alice sends the message + signature.
- **Verifying:** Bob hashes the received message himself. He decrypts Alice's signature using Alice's PUBLIC key. If the two hashes match, the message is authentic and unmodified.

Key Insight: Why encrypt only the hash and not the whole message? Efficiency. Asymmetric encryption is slow; hashing is fast. The tiny fixed-size hash is cheap to sign.

Certificates and PKI

How does Alice know Bob's public key really is Bob's, and not an attacker's? Through a **Public Key Infrastructure (PKI)**. A trusted **Certificate Authority (CA)** vouches for the binding between an identity and a public key by signing a **digital certificate**.

Your browser trusts a set of root CAs (Let's Encrypt, DigiCert, etc.). When you visit an HTTPS site, the server presents a certificate signed by a CA. If your browser trusts that CA, it accepts the server's public key. This is what makes HTTPS work.

8.6 TLS/SSL — Securing the Transport

TLS (Transport Layer Security), formerly known as SSL, is the protocol that adds encryption, authentication, and integrity to TCP. HTTPS is HTTP over TLS. Also used for secure email (SMTPS, IMAPS), secure DNS (DoT), VPNs (OpenVPN).

TLS Handshake (Simplified)

- **1. Client Hello:** Client offers supported TLS versions, cipher suites, random number.
- **2. Server Hello:** Server picks the version and cipher, sends its certificate (containing public key), sends its random number.
- **3. Client verifies certificate** using its trusted CAs.
- **4. Key exchange:** Client generates a pre-master secret and encrypts it with server's public key (RSA), OR both derive a shared secret from a Diffie-Hellman exchange.
- **5. Session keys derived** from pre-master + both randoms. Both sides now share a symmetric key.
- **6. Encrypted communication** continues using the symmetric session key.

TLS 1.3 (2018) streamlined this to fewer round trips and removed old, weak options. Modern browsers prefer TLS 1.3.

8.7 Firewalls, IDS, and Practical Defenses

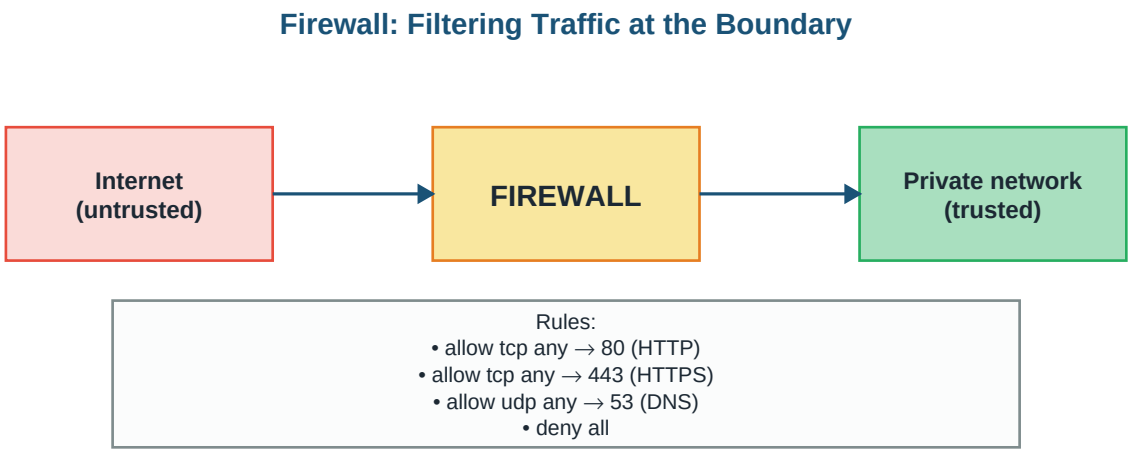


Figure: Firewall — filters traffic based on rules

Firewalls

- **Packet-filtering firewall:** Inspects each packet's IP addresses, ports, protocol. Allows or drops based on rules. Fast but doesn't understand context (e.g., can't tell an ACK is part of an established connection).
- **Stateful firewall:** Tracks connection state. Knows that an inbound packet is a reply to an outbound request. More permissive but more secure.
- **Application-layer firewall:** Inspects the actual application-layer data. Can enforce rules like "block HTTP requests containing SQL injection patterns". Slower but more powerful.
- **Web Application Firewall (WAF):** Application-layer firewall specifically for HTTP/HTTPS. Sits in front of web servers, blocks common attacks.

IDS and IPS

- **Intrusion Detection System (IDS):** Monitors traffic and alerts on suspicious patterns. Doesn't block — just tells you something bad might be happening.
- **Intrusion Prevention System (IPS):** IDS that actively blocks bad traffic. More responsive but with risk of false positives blocking legitimate users.

VPNs

A **Virtual Private Network** creates an encrypted tunnel between two endpoints over an untrusted network. Uses: employees securely accessing office resources from home; privacy from ISPs; bypassing regional content blocks. Common protocols: **OpenVPN**, **WireGuard**, **IPsec**.

Appendix

Quick Reference Formulas & Key Points

One-page revision reference. All the essential formulas, terms, and comparisons in one place — perfect for a last look before exam or interview.

A.1 Reference Models Cheat Sheet

OSI Layers (top-down): Application, Presentation, Session, Transport, Network, Data Link, Physical

Mnemonic: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing

TCP/IP (top-down): Application, Transport, Internet, Data Link, Physical

PDU per layer: App=Data, Transport=Segment, Network=Packet, Data Link=Frame, Physical=Bit

A.2 Physical Layer Formulas

```
Bandwidth-Delay Product = Bandwidth × RTT
(max data in flight at any moment)

Transmission delay = frame_size / bandwidth
Propagation delay = distance / propagation_speed
Total delay = t_trans + t_prop + t_proc + t_queue

Speed of light in fiber ≈ 2 × 10■ m/s
Speed of light in copper ≈ 2 × 10■ m/s
```

A.3 Error Detection

- **Parity:** 1 extra bit, detects only odd-number errors.
- **Checksum:** Sum of data words, one's complement. Used by TCP, UDP, IP.
- **CRC:** Polynomial division. CRC-32 catches essentially all real-world errors.
- **Hamming Code:** Adds ■log■ n■ + 1 bits, corrects single-bit errors.

A.4 ALOHA / MAC Protocols

```
Pure ALOHA throughput: S = G × e■⁻²■ (max ≈ 0.184 at G = 0.5)
Slotted ALOHA throughput: S = G × e■■ (max ≈ 0.368 at G = 1.0)

CSMA/CD: sense → transmit → detect collision → jam → binary exp backoff
```

A.5 Sliding Window Constraints

```
For n-bit sequence numbers:
Go-Back-N: W_sender ≤ 2■ − 1
Selective Repeat: W_sender + W_receiver ≤ 2■
(equivalently W ≤ 2■■¹ for symmetric)
```

GBN: Receiver only accepts in-order frames. Sender re-sends everything from lost frame.

SR: Receiver buffers out-of-order. Sender re-sends only lost frames.

A.6 IP Addressing

```
IPv4: 32 bits, dotted decimal, ~4.3 × 10■ addresses
IPv6: 128 bits, hex groups, ~3.4 × 10³■ addresses

CIDR /n → n network bits, (32 − n) host bits
Total addresses in /n = 2^(32 − n)
Usable hosts = 2^(32 − n) − 2 (subtract network + broadcast)
```

Private IPv4 ranges (RFC 1918):

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

Loopback: 127.0.0.0/8 | **APIPA link-local:** 169.254.0.0/16 | **Multicast:** 224.0.0.0/4

A.7 Routing Protocols

Type	Algorithm	Protocols
Distance-Vector	Bellman-Ford	RIP (max 15 hops)
Link-State	Dijkstra	OSPF, IS-IS
Path-Vector	Path advertisement	BGP (between ASes)

Bellman-Ford update: cost(D) = min over neighbors N of (cost(N) + N's cost to D)

Fixes for count-to-infinity: split horizon, poison reverse, hold-down timers

A.8 Well-Known Port Numbers

Port	Protocol	Purpose
20/21	FTP	File transfer
22	SSH	Secure shell
23	TELNET	Insecure remote login
25	SMTP	Email sending
53	DNS	Name resolution (UDP mostly)
67/68	DHCP	IP assignment (server/client)

Port	Protocol	Purpose
80	HTTP	Web (unencrypted)
110	POP3	Mail retrieval
143	IMAP	Mail sync
443	HTTPS	Web (TLS)
3306	MySQL	Database
5432	PostgreSQL	Database

A.9 TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (ACKs, retransmit)	Best effort
Ordering	In-order delivery	Not guaranteed
Flow control	Yes	No
Congestion control	Yes	No
Header size	20+ bytes	8 bytes
Speed	Slower	Faster
Use cases	HTTP, SSH, email, file transfer	DNS, VoIP, gaming, streaming

A.10 TCP Connection

3-way handshake: SYN → SYN+ACK → ACK

4-way termination: FIN → ACK → FIN → ACK

TIME_WAIT: 2 × MSL (Maximum Segment Lifetime)

SYN flood defense: SYN cookies

A.11 TCP Congestion Control

AIMD: Additive Increase (+1 MSS per RTT), Multiplicative Decrease (÷ 2 on loss)

Slow Start: cwnd doubles per RTT until it reaches ssthresh

Congestion Avoidance: cwnd += 1 MSS per RTT (linear)

Timeout (Tahoe): ssthresh = cwnd/2, cwnd = 1 MSS, restart with slow start

Fast Recovery (Reno): On 3 dup ACKs, ssthresh = cwnd/2, cwnd = ssthresh, keep going

A.12 DNS

Hierarchy: Root (.) → TLD (com, org, ...) → 2nd-level (google.com) → subdomains

Recursive query: Client → local resolver (resolver does all work)

Iterative query: Local resolver → root → TLD → authoritative

Record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (name server), TXT, PTR

Default port: 53 (UDP for queries, TCP for large responses/zone transfers)

A.13 HTTP Status Codes

- **1xx Informational:** 100 Continue
- **2xx Success:** 200 OK, 201 Created, 204 No Content
- **3xx Redirection:** 301 Moved Permanently, 304 Not Modified
- **4xx Client Error:** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
- **5xx Server Error:** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

A.14 Security Building Blocks

- **CIA Triad:** Confidentiality, Integrity, Availability
- **Symmetric encryption:** Same key both ends. Fast. Examples: AES, ChaCha20.
- **Asymmetric encryption:** Public + private key pair. Slow but solves key distribution. Examples: RSA, ECC.
- **Hybrid encryption:** Asymmetric to exchange symmetric key, then symmetric. Used by TLS.
- **Hash functions:** One-way, fixed output. Use SHA-256 or better. Avoid MD5, SHA-1.
- **Digital signature:** Hash + encrypt hash with sender's PRIVATE key. Verify with PUBLIC key.
- **Certificate:** Public key + identity signed by a CA. Foundation of HTTPS.

A.15 Common Attacks & Defenses

Attack	Defense
Eavesdropping	Encryption (TLS)
Man-in-the-middle	Certificates + mutual auth
Replay	Nonces, timestamps, sequence numbers
DoS / DDoS	Rate limiting, upstream filtering, CDN
DNS spoofing	DNSSEC
ARP spoofing	Static ARP, DHCP snooping
SYN flood	SYN cookies
Phishing	User education, 2FA, DMARC

All the best for GATE & Placements — you've got this!